

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке отказоустойчивой микросервисной веб-системы для управления заявками на трансграничные финансовые переводы. Целью исследования является создание безопасной CRM-платформы, обеспечивающей строгую изоляцию данных посредством многоуровневой модели ролевого доступа.

В ходе выполнения работы проведен детальный анализ специфики финансовых транзакций, спроектирована распределенная архитектура программного комплекса и реализован сложный конечный автомат управления жизненным циклом сделок. Серверная инфраструктура разработана с использованием стека Python (FastAPI) и микросервиса на Go, выполняющего функции API Gateway с интегрированным алгоритмом ограничения частоты запросов (Sliding Window) на базе кластера Redis. Клиентская часть построена на фреймворке Next.js. Интегрированы брокер сообщений RabbitMQ и кластер Centrifugo для обеспечения защищенного обмена сообщениями в реальном времени. Развертывание платформы автоматизировано через конвейер GitLab CI/CD в среде Docker Swarm.

Практическая ценность результатов исследования заключается в автоматизации процессов верификации заявок, снижении операционных рисков и повышении прозрачности работы профильных подразделений (комплаенс-контроля, юристов, финансистов, менеджеров).

Общий объем работы составляет страниц ...

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И МЕТОДОВ ПОСТРОЕНИЯ ФИНАНСОВЫХ ВЕБ-СИСТЕМ	8
1.1. Современное состояние и проблемы автоматизации трансграничных финансовых операций	8
1.2. Классификация бизнес-процессов и ролевых моделей в системах управления заявками	11
1.3. Сравнительный анализ существующих CRM-решений и их архитектурных недостатков	16
1.4. Обоснование выбора микросервисной архитектуры для финансовых платформ	18
1.5. Формирование требований к разрабатываемой веб-системе	22
Выводы по первой главе	26
ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И БИЗНЕС-ЛОГИКИ СЕРВИСА УПРАВЛЕНИЯ ЗАЯВКАМИ	29
2.1. Разработка общей архитектуры программного комплекса и топологии сети	29
2.2. Обоснование выбора стека технологий реализации	32
2.3. Проектирование микросервиса API Gateway и подсистемы защиты от перегрузок	37
2.3.1. Механизм аутентификации и валидации JWT-токенов	38
2.3.3. Обоснование и проектирование алгоритма ограничения частоты запросов (Rate Limit)	39
2.3.4. Маршрутизация и агрегация данных	42
Выводы по второй главе	42
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ	44
3.1. Реализация серверной части бизнес-логики и REST API интерфейсов	44
3.2. Реализация шлюза запросов и алгоритма Rate Limit	45
3.4. Разработка клиентского веб-интерфейса и ролевого рендеринга	49

3.4. Тестирование программного комплекса и оценка отказоустойчивости....	54
3.4.1. Функциональное и интеграционное тестирование	55
3.4.2. Нагрузочное тестирование и верификация алгоритма Rate Limit	55
3.4.3. Оценка отказоустойчивости (Fault Tolerance).....	57
3.4.5. Итоги тестирования.....	58
Выводы по третьей главе	59
ЗАКЛЮЧЕНИЕ.....	62
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	63
ПРИЛОЖЕНИЕ А	65

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

ВВЕДЕНИЕ

Актуальность темы. Широкое распространение облачных технологий сопровождается ростом числа утечек данных, значительная часть которых связана с архитектурными недостатками систем безопасности, слабыми механизмами аутентификации и отсутствием изолированного хранения криптографических ключей. Существующие облачные сервисы часто имеют доступ и к данным, и к ключам дешифрования, что повышает риск их компрометации. В этих условиях возрастает потребность в создании сервисов, обеспечивающих криптографическую защиту данных и многофакторную, включая беспарольную, аутентификацию.

Суть проблемной ситуации. Современные публичные облачные хранилища не обеспечивают полного разделения сфер ответственности между пользователем и провайдером, что делает невозможным гарантирование конфиденциальности данных. Отсутствие архитектурной изоляции и использование паролей как основного фактора аутентификации увеличивают вероятность несанкционированного доступа. Решение проблемы требует разработки системы, исключающей доступ провайдера к ключам и обеспечивающей современную аутентификацию.

Объект исследования: защищённые распределённые информационные системы и облачные хранилища данных.

Предмет исследования: методы криптографической защиты данных, механизмы аутентификации пользователей и алгоритмы управления ключами в веб-приложениях.

Цель работы: разработка защищённого облачного сервиса хранения файлов с двухфакторной и беспарольной аутентификацией (WebAuthn/Passkey) и серверным шифрованием с изолированным хранением ключей.

Методы исследования: системный и сравнительный анализ, моделирование информационных процессов, объектно-ориентированное

проектирование, методы симметричной криптографии, функциональное и нагрузочное тестирование.

Задачи исследования: Для достижения поставленной цели необходимо решить следующие задачи:

Провести комплексный анализ актуальных угроз информационной безопасности облачных сервисов хранения данных, выявить типичные векторы атак и архитектурные уязвимости существующих решений.

1) сформулировать функциональные и нефункциональные требования к защищённому облачному хранилищу, определить критерии безопасности и обосновать выбор технологического стека для реализации серверной и клиентской частей системы;

2) спроектировать микросервисную архитектуру веб-сервиса с чётким разделением компонентов клиентской и серверной частей, обеспечивающую изоляцию модулей хранения данных, управления секретами и аутентификации пользователей;

3) реализовать модуль многофакторной аутентификации, включающий поддержку традиционной двухфакторной аутентификации на основе одноразовых паролей (TOTP) и современной беспарольной аутентификации с использованием стандарта WebAuthn/Passkey;

4) разработать подсистему серверного шифрования файлов с применением симметричных криптографических алгоритмов и реализовать интеграцию с внешним хранилищем секретов HashiCorp Vault для изолированного управления ключами шифрования;

5) создать пользовательский интерфейс веб-приложения на базе современного фреймворка, обеспечивающий интуитивное взаимодействие с функциями загрузки, скачивания, управления файлами и настройками безопасности;

6) провести комплексное тестирование разработанного прототипа, включающее функциональное тестирование сценариев использования, нагрузочное тестирование производительности криптографических

операций и количественную оценку безопасности с расчётом коэффициента снижения риска по методике ALE.

Первая глава посвящена анализу угроз и существующих решений.

Вторая глава содержит проектирование архитектуры сервиса и подсистемы управления ключами.

Третья глава раскрывает программную реализацию, механизмы аутентификации и результаты тестирования.

Достоверность результатов обеспечивается применением стандартов и рекомендаций в области информационной безопасности, использованием проверенных криптографических библиотек и функциональным тестированием прототипа.

Научная ценность заключается в обосновании архитектуры, обеспечивающей изоляцию хранения данных и криптографических ключей, а также в применении комплексного подхода к аутентификации, сочетающего традиционные и современные методы.

Практическая ценность состоит в создании прототипа, пригодного для использования как основы корпоративных или персональных защищённых хранилищ, поддерживающего контейнеризацию и масштабирование и обеспечивающего более высокий уровень конфиденциальности данных.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И МЕТОДОВ ПОСТРОЕНИЯ ФИНАНСОВЫХ ВЕБ-СИСТЕМ

1.1. Современное состояние и проблемы автоматизации трансграничных финансовых операций

На текущем этапе развития мировой экономики сектор финансовых технологий (FinTech) демонстрирует беспрецедентные темпы роста. Данная тенденция обусловлена необратимыми процессами макроэкономической глобализации, расширением географии международной торговли и острой необходимостью обеспечения бесперебойного, высокоскоростного движения капиталов между различными юрисдикциями. Традиционные механизмы межбанковских трансграничных переводов, базирующиеся на классических системах обмена финансовыми сообщениями (таких как SWIFT) и использовании сложных цепочек корреспондентских счетов, все чаще демонстрируют свою неэффективность в условиях новых рыночных реалий. К числу их основных недостатков традиционно относят высокие транзакционные издержки, длительные сроки клиринга и расчетов (достигающие нескольких банковских дней), а также высокую степень уязвимости перед геополитическими и санкционными рисками, что приводит к блокировкам или заморозке средств конечных пользователей.

В связи с вышеизложенным, на глобальном финансовом рынке сформировался и стремительно масштабируется устойчивый спрос на альтернативные расчетные маршруты. Современная парадигма трансграничных переводов включает в себя не только классические операции с государственными валютами (модели FIAT_FIAT), но и инновационные гибридные схемы, предполагающие использование цифровых активов и стейблкоинов в качестве транзитного инструмента обмена ценностью. В архитектуре проектируемой системы такие маршруты классифицируются как направления CRYPTO_FIAT (конвертация цифровых активов в фиатные средства в стране получения) и FIAT_CRYPTO (приобретение цифровых активов за фиатные средства с их последующим переводом). Каждое из этих направлений характеризуется уникальным набором требований к

юридическому оформлению, методологии оценки рисков, процедурам комплаенс-контроля и строгости документарного сопровождения.

Процесс проведения подобных комплексных сделок представляет собой многоуровневую цепочку взаимодействий. В ней принимают участие не только непосредственный заказчик (клиент), но и множество агентов-посредников, а также субагентов (провайдеров ликвидности), которые обеспечивают физическое или цифровое наличие средств в конечных точках маршрута. Несмотря на активное внедрение информационных технологий в финансовый сектор, автоматизация столь многоуровневых операций сопряжена с рядом фундаментальных барьеров и нерешенных проблем:

1. Многоэтапность и задержки в процедурах согласования и верификации. Каждая финансовая транзакция, в особенности связанная с международным движением капитала, требует неукоснительного соблюдения политик KYC (Know Your Customer) и AML (Anti-Money Laundering). Проверка учредительных документов юридических лиц (тип `COMPANY_LEGAL_ENTITY`), анализ правового статуса индивидуальных предпринимателей (`COMPANY_INDIVIDUAL_ENTREPRENEUR`), а также глубокий аудит источников происхождения средств занимают колоссальное количество времени. В отсутствие централизованной и защищенной информационной среды обмен критически важными документами (инвойсами, договорами, актами приема-передачи) осуществляется по разрозненным каналам связи, что неизбежно приводит к информационному разрыву и критическим задержкам в бизнес-цикле.

2. Критическое влияние человеческого фактора и операционные риски. При ручном или полуавтоматизированном управлении статусами заявок экспоненциально возрастает риск совершения критических ошибок. К таковым относятся: инициация перевода средств до момента полного подписания закрывающих юридических документов, нарушение утвержденной маршрутизации заявки, а

также некорректный расчет или выплата комиссионных вознаграждений агентам. Подобные инциденты влекут за собой прямые финансовые убытки и штрафные санкции со стороны регуляторов.

3. Проблема обеспечения конфиденциальности и строгой изоляции данных. В финансовых цепочках участвуют независимые контрагенты. Возникает феномен так называемой «информационной асимметрии». Раскрытие информации о конечных комиссиях платформы, данных первоначального отправителя или деталях договоров субагентов неавторизованному звену цепи является грубым нарушением коммерческой тайны. Существующие системы зачастую не позволяют гибко скрывать отдельные атрибуты сущностей от конкретных ролей, предоставляя либо полный доступ к карточке сделки, либо полностью блокируя его.

4. Отсутствие гибкости в управлении исключениями и нештатными ситуациями. Процесс проведения сделки не всегда является линейным. Зачастую возникают спорные ситуации: расхождения в зафиксированном курсе конвертации, недополучение заявленного объема средств (Volume), либо претензии к качеству сопроводительной документации. Возникновение подобных разногласий требует немедленного перевода сделки в статус апелляции (DEAL_STATUS_AGENT_APPEAL) с автоматической приостановкой всех расчетных процессов и привлечением профильных специалистов (юристов, финансистов). Большинство коробочных систем не способны обработать такие "боковые ветви" бизнес-процесса без нарушения общей логики работы.

Исходя из проведенного анализа, представляется очевидным, что современное состояние предметной области диктует безальтернативную необходимость создания специализированных, узкопрофильных программных комплексов. Подобные комплексы должны не просто выступать

хранилищем данных, а функционировать как строгий математический конечный автомат (State Machine), который на алгоритмическом уровне блокирует любые действия, не предусмотренные регламентом для текущего статуса сделки и текущей роли пользователя.

1.2. Классификация бизнес-процессов и ролевых моделей в системах управления заявками

Для обеспечения высочайшего уровня информационной безопасности, предотвращения внутренних угроз (фрода) и поддержания прозрачности управления заявками, фундаментальным требованием является проектирование и внедрение бескомпромиссной модели разграничения прав доступа. В теории информационной безопасности существует несколько базовых парадигм: дискреционное (DAC), мандатное (MAC), атрибутное (ABAC) и ролевое управление доступом (RBAC). Проведенный теоретический анализ показал, что для корпоративных систем с четко регламентированными бизнес-процессами наиболее эффективной и масштабируемой является модель Role-Based Access Control (RBAC). Данная концепция основана на принципе, что права на выполнение операций в системе (чтение, запись, изменение статуса, скачивание файла) назначаются не конкретным физическим пользователям, а абстрактным ролям, которые эти пользователи исполняют в соответствии со штатным расписанием организации.

В рамках разрабатываемой CRM-системы бизнес-процессы классифицируются по нескольким ключевым доменам, каждый из которых обслуживается определенной группой ролей. К основным доменам относятся: процессы первичной регистрации и верификации контрагентов, процессы согласования и генерации юридически значимой документации (договоров, инвойсов, актов), процессы сверки и подтверждения финансовых объемов, а также процессы арбитража и разрешения споров.

Для каждого из выделенных доменов была сформирована детальная матрица ответственности, опирающаяся на принцип минимальных привилегий (Principle of Least Privilege). В системе предусмотрены следующие ключевые функциональные роли:

- Менеджер (Manager). Является фронт-офисным сотрудником и ответственным за первичное взаимодействие с клиентом. В задачи менеджера входит сбор базовой информации, формирование тела заявки, загрузка первичных документов от клиента и сопровождение сделки по этапам. Ограничения: менеджер имеет доступ исключительно к тем заявкам, которые закреплены за ним лично. Он категорически лишен прав на прямое изменение финансовых метрик сделки, не может самостоятельно подтверждать прохождение комплаенс-контроля и не имеет доступа к внутренней коммерческой информации субагентов.

- Специалист подразделения финансового мониторинга. Его исключительной зоной ответственности является проведение процедур AML/KYC. Комплаенс-офицер анализирует загруженные уставные документы компаний, проверяет их по санкционным спискам и базам данных неблагонадежных контрагентов. Архитектурно заложено, что без цифровой визы (установки статуса верификации) со стороны комплаенс-офицера переход жизненного цикла заявки на этапы, разрешающие генерацию ордеров и движение средств, алгоритмически невозможен.

- Юрист. Выполняет функции правового аудита и сопровождения. Бизнес-процесс подразумевает, что на определенных этапах (например, при формировании `AGREEMENT_FOR_SUBAGENT` или `ORDER_FROM_CUSTOMER`) требуется верификация корректности составления контрактов. Юрист проверяет и подписывает шаблоны документов. Кроме того, роль юриста является критической при переходе сделки в статус апелляции, где требуется правовое урегулирование возникших претензий между участниками транзакции.

- Финансист. Наиболее привилегированная роль в контексте управления денежными потоками. Финансист работает в бэк-офисе платформы и

отвечает за сверку балансов. Именно этот участник подтверждает фактическое поступление заявленного объема средств (в фиатном эквиваленте на банковские счета или в криптовалютном на холодные/горячие кошельки). Только после того как финансист переводит систему в статус **подтверждения** **объема** (DEAL_SUBSTATUS_AGENT_RECEIVING_VOLUME), бизнес-логика допускает переход к этапам формирования и подписания закрывающих актов выполненных работ.

- Контролер (Аудитор). Выполняет функции независимого внутреннего аудита. Данная роль наделена широкими правами на просмотр данных (глобальный read-only доступ) по всем сделкам, филиалам и агентам. Контролер отслеживает KPI менеджеров, проверяет соблюдение временных регламентов (SLA) при обработке заявок и выявляет аномалии в цепочках транзакций.

- Администратор. Техническая роль, обладающая полнотой прав на конфигурирование системных справочников, управление учетными записями, принудительный сброс паролей и назначение ролей другим пользователям. Администратор не участвует в операционном цикле сделок, но обеспечивает работоспособность инфраструктуры.

Для наглядного представления и структурирования информационных потоков, отражающих взаимодействие описанных ролей с функциональными модулями системы, в ходе проектирования была разработана диаграмма прецедентов (Use Case Diagram), представленная на рисунке 1.1.

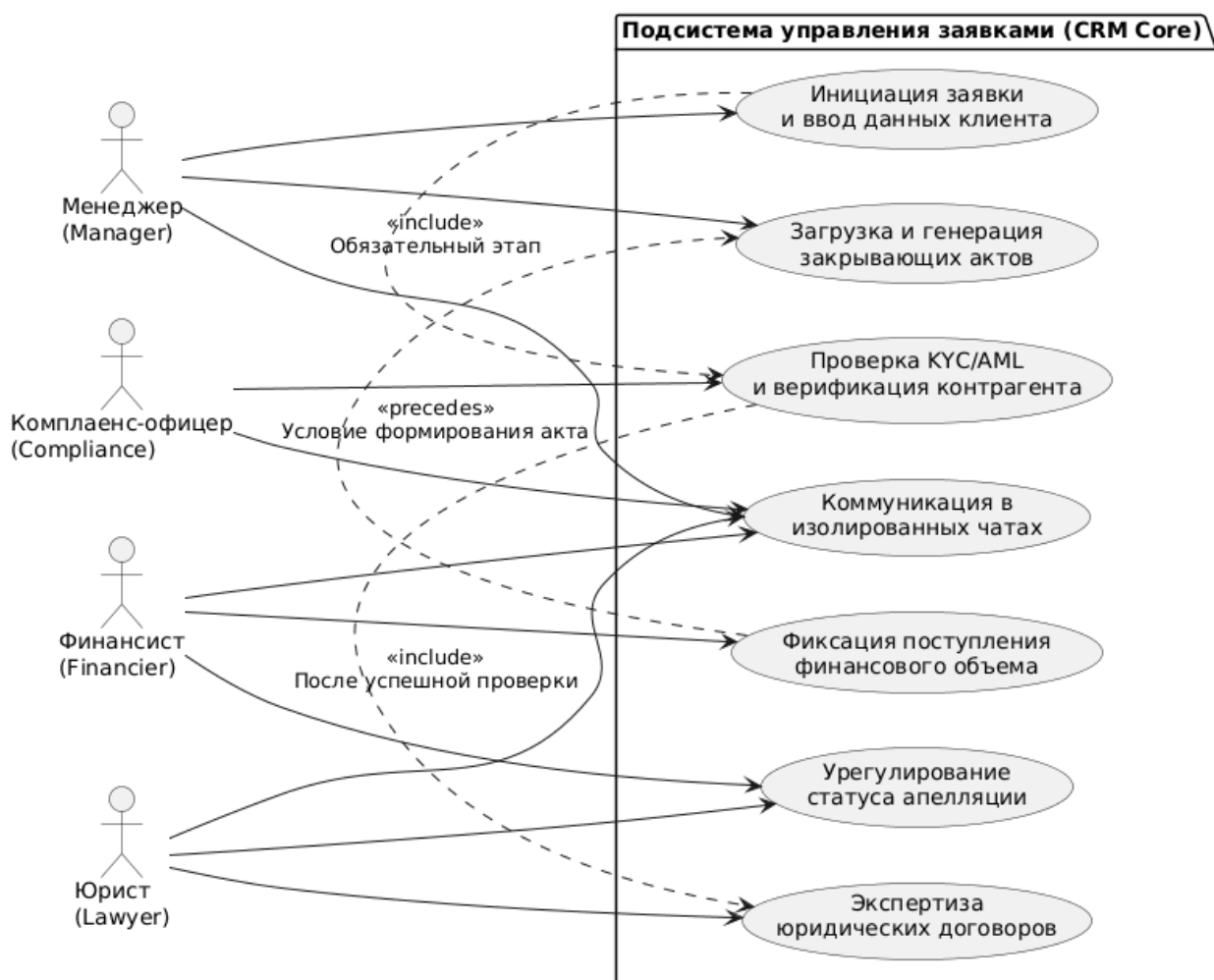


Рисунок 1.1 — Диаграмма прецедентов (Use Case) ролевого взаимодействия в системе

Для более глубокого понимания процессуальной динамики необходимо рассмотреть не только статические разрешения, но и алгоритм передачи ответственности (токен управления) от одной роли к другой. В контексте жизненного цикла заявки, как только менеджер переводит сделку в статус проверки инвойса, интерфейс менеджера переходит в режим ожидания (read-only для текущего шага), а у комплаенс-офицера в специализированном рабочем пространстве (дашборде) появляется соответствующее уведомление и активируются элементы управления.

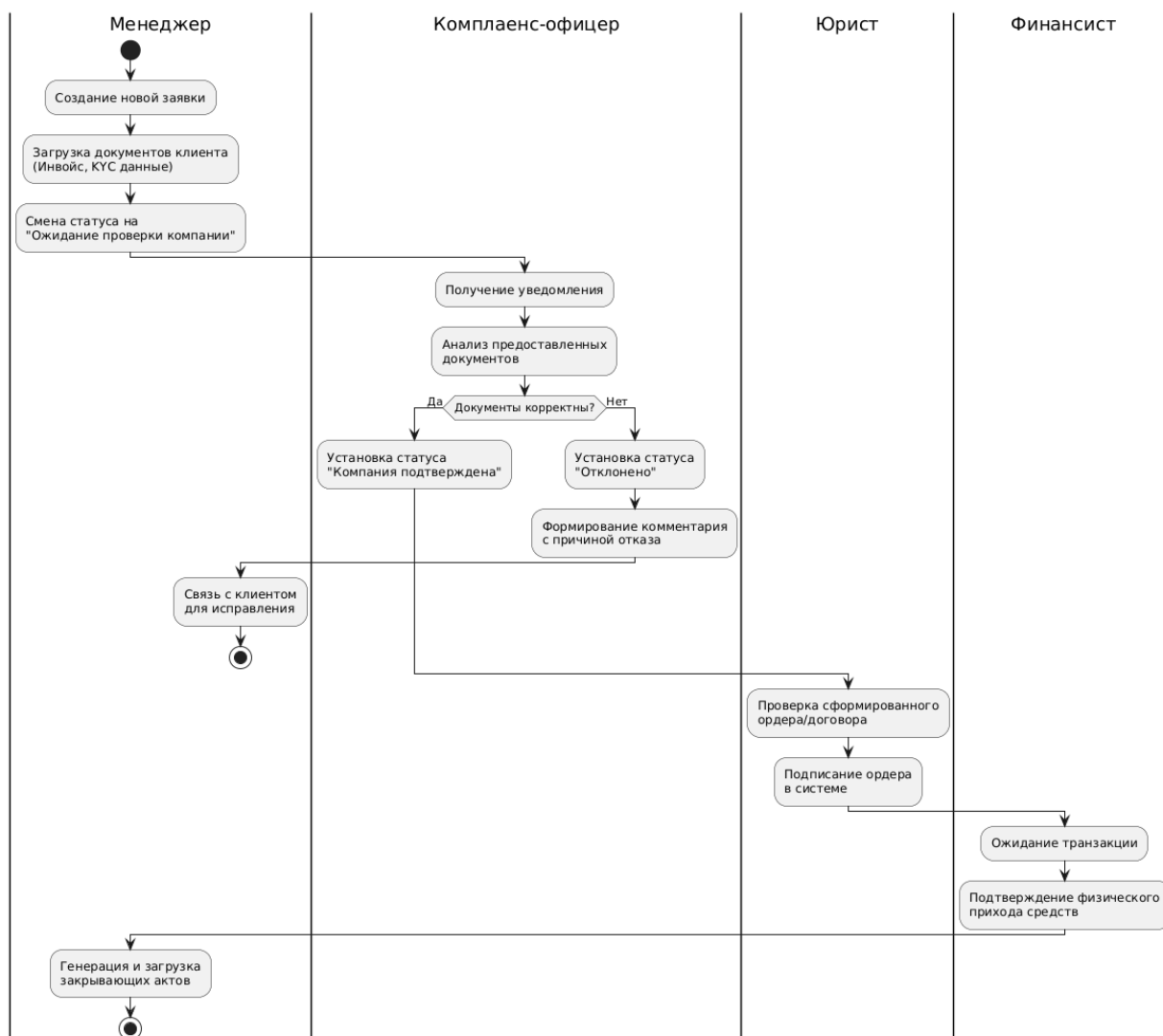


Рисунок 1.2 — Диаграмма деятельности (Activity Diagram) ролевой модели при обработке заявки

Как демонстрирует рисунок 1.2, обязанности и точки принятия решений строго сегментированы и выстроены в неразрывную хронологическую цепь. Подобная классификация бизнес-процессов на фундаментальном уровне исключает возникновение конфликта интересов, минимизирует риски преднамеренных или случайных деструктивных действий и обеспечивает высокую степень прозрачности (Traceability) каждой финансовой операции для последующего аудита. Таким образом, внедрение детально проработанной RBAC-модели является не просто требованием

безопасности, а базовым архитектурным принципом, на котором строится вся логика функционирования программного комплекса.

1.3. Сравнительный анализ существующих CRM-решений и их архитектурных недостатков

При проектировании корпоративной информационной среды для финансового сектора закономерно возникает необходимость оценки целесообразности внедрения готовых коммерческих продуктов (COTS — Commercial Off-The-Shelf) в противовес разработке собственного, узкоспециализированного (Custom) решения. Для проведения объективной и всесторонней оценки был осуществлен сравнительный анализ наиболее востребованных на рынке платформ класса CRM: AmoCRM, Bitrix24, а также решений корпоративного (Enterprise) уровня, таких как Salesforce и Microsoft Dynamics. Оценка производилась по ряду критических критериев, специфицированных предметной областью: гибкость настройки и глубина вложенности конечных автоматов (State Machines), возможность обеспечения строгой изоляции финансовых данных между контрагентами (субагентами и агентами), а также готовность базовой архитектуры к высоким нагрузкам при обработке потоковых данных в режиме реального времени.

Универсальные SaaS-решения для массового рынка (AmoCRM, Bitrix24).

Архитектура данных систем исторически спроектирована для обслуживания классических процессов продаж (Sales Pipeline) и управления лидами. Их базовая бизнес-логика опирается на линейное, последовательное перемещение сущности (карточки сделки) по этапам так называемой «воронки продаж» (например: «Первичный контакт», «Переговоры», «Выставление счета», «Успешно реализовано»). В контексте автоматизации трансграничных финансовых операций подобная линейная модель выявляет свою фундаментальную несостоятельность. Жизненный цикл сложной финансовой транзакции подразумевает одновременное, параллельное

существование и отслеживание сразу нескольких статусных моделей. В частности, в рамках одной заявки могут параллельно функционировать независимые, но взаимосвязанные конечные автоматы: статусы обработки лида (DealStatusLead), статусы взаимодействия с агентом (DealStatusAgent) и статусы взаимодействия с конечным провайдером ликвидности — субагентом (DealStatusSubagent).

Кроме того, в универсальных системах крайне затруднительно, а зачастую невозможно без использования небезопасных программных «костылей», реализовать сложную, многомерную матрицу прав доступа на уровне отдельных полей или блоков внутри одной сущности. Например, категорически недопустимо, чтобы менеджер, ведущий клиента, имел доступ к информации о размере чистой прибыли платформы или к банковским реквизитам субагента, обеспечивающего фиатный шлюз. Универсальные CRM-решения предоставляют преимущественно бинарный доступ (доступ к сделке либо есть, либо его нет), что прямо нарушает требования банковской и коммерческой тайны.

Enterprise-платформы корпоративного уровня (Salesforce, Microsoft Dynamics).

Платформы данного класса обладают значительно более широкими возможностями для кастомизации бизнес-логики, в том числе за счет встроенных интерпретируемых языков программирования (например, Apex в среде Salesforce). Однако их архитектура, являющаяся по своей сути высокоуровневым монолитом, накладывает жесткие ограничения на масштабируемость специфических узлов, требующих нестандартных протоколов взаимодействия.

В проектируемой системе критическим требованием является наличие изолированных чатов (внутренних, клиентских, апелляционных) с использованием протокола WebSockets для мгновенного обмена сообщениями. Интеграция высоконагруженных постоянных соединений (Persistent Connections) в монолитную среду традиционной CRM-системы

сопряжена с рисками исчерпания пула потоков балансировщиков нагрузки и нестабильной работой интерфейса. Более того, лицензионная политика Enterprise-решений, тарифицирующая количество API-запросов и объем хранимых данных, делает экономически нерентабельным процессинг микрособытий (например, событий набора текста в чате или промежуточных проверок комплаенс-контроля).

Сравнительный анализ позволяет сделать обоснованный вывод о том, что существующие на рынке коробочные монолитные решения не способны удовлетворить совокупность требований к масштабируемости, безопасности и сложности бизнес-логики. Попытки адаптации подобных систем под жесткие рамки финансового процессинга приводят к перегрузке программного кода плагинами, неконтролируемой связанности модулей и, как следствие, катастрофическому снижению общей отказоустойчивости. Таким образом, разработка собственной распределенной системы является единственным технологически и экономически обоснованным путем решения поставленной задачи.

1.4. Обоснование выбора микросервисной архитектуры для финансовых платформ

Определение базового архитектурного паттерна является критическим этапом проектирования любой высоконагруженной системы. В современной программной инженерии концептуальный выбор зачастую сводится к противостоянию двух фундаментальных парадигм: монолитной архитектуры и архитектуры микросервисов (Microservices Architecture, MSA).

При монолитном подходе все функциональные модули системы (модуль авторизации, бизнес-логика управления заявками, подсистема генерации документов, модуль коммуникации) компилируются и развертываются как единый, неделимый процесс. Основным недостатком данной парадигмы в контексте финансовых платформ является сильная связанность (Tight Coupling) компонентов. При возникновении программной ошибки (Memory Leak) в модуле генерации PDF-договоров, аварийно завершается весь процесс, что приводит к остановке процессинга транзакций для всех

пользователей системы. Кроме того, монолит крайне неэффективно масштабируется: при резком увеличении нагрузки на подсистему чатов приходится дублировать (масштабировать) всё приложение целиком, включая тяжеловесные модули, не испытывающие нагрузки, что приводит к нерациональному расходу вычислительных ресурсов.

В противовес монолиту, микросервисная архитектура предполагает декомпозицию приложения на набор независимых, слабо связанных сервисов, каждый из которых выполняет строго определенную бизнес-функцию (Bounded Context) и развертывается автономно. Для разрабатываемой системы управления финансовыми заявками микросервисный подход предоставляет ряд неоспоримых преимуществ:

1. Изолированность отказов (Fault Isolation). В случае недоступности или сбоя микросервиса обмена сообщениями (чатов), базовое ядро системы (CRM Core), отвечающее за смену статусов сделок и движение средств, продолжит функционировать в штатном режиме.

2. Гетерогенность технологического стека (Polyglot Persistence/Programming). Микросервисный подход позволяет подбирать наиболее оптимальный язык программирования и тип базы данных для конкретной задачи. Так, высокопроизводительный шлюз для обработки входящего трафика целесообразно реализовать на компилируемом языке с легковесными потоками (например, Go), в то время как сложную бизнес-логику конечных автоматов и валидацию данных эффективнее реализовывать на языках с мощной экосистемой веб-фреймворков (например, Python/FastAPI с использованием PostgreSQL).

3. Независимое масштабирование (Independent Scaling). В периоды пиковых нагрузок инфраструктура оркестрации контейнеров (например, Docker Swarm) позволяет динамически увеличивать количество экземпляров (реплик) только тех

микросервисов, которые испытывают перегрузку, не затрагивая остальные компоненты.

Для управления сложностью распределенной системы и обеспечения ее безопасности в рамках микросервисной парадигмы активно применяются два ключевых паттерна: API Gateway и Событийно-ориентированная архитектура (Event-Driven Architecture, EDA).

Паттерн API Gateway выступает в роли единой точки входа (Single Entry Point) для всех клиентских запросов. На уровне данного шлюза концентрируются сквозные (Cross-cutting) задачи, такие как терминация SSL-соединений, первичная аутентификация по JWT-токенам, маршрутизация трафика (Reverse Proxy) и, что критически важно, ограничение частоты запросов (Rate Limiting). Интеграция алгоритма Sliding Window на уровне API Gateway позволяет эффективно фильтровать нелегитимный трафик, защищая внутренние сервисы от DDoS-атак и истощения ресурсов.

Паттерн Event-Driven Architecture решает проблему синхронного блокирования при межсервисном взаимодействии. Вместо прямых HTTP-вызовов, которые могут привести к каскадным сбоям, микросервисы общаются посредством публикации и подписки на события через отказоустойчивые брокеры сообщений (например, кластер RabbitMQ).

На рисунке 1.3 представлена структурная схема, демонстрирующая преимущества спроектированной микросервисной парадигмы с применением API Gateway и брокера сообщений.

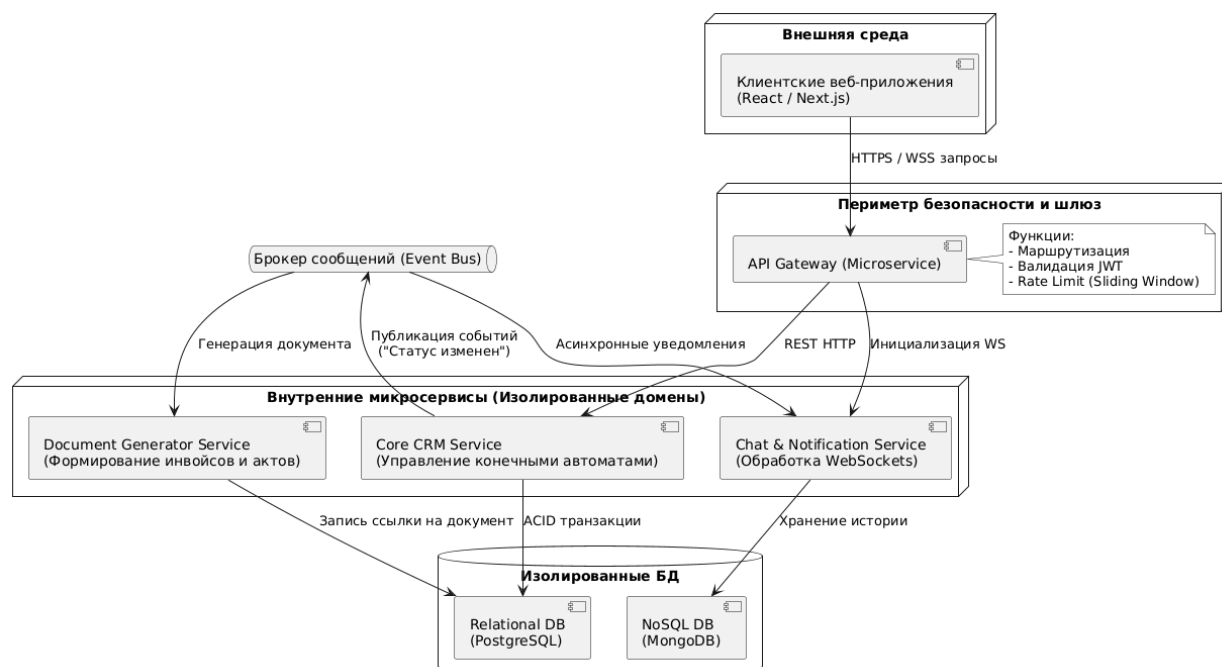


Рисунок 1.3 — Модель взаимодействия компонентов в микросервисной событийно-ориентированной архитектуре

Анализ теоретических аспектов и практических паттернов проектирования, проиллюстрированных на рисунке 1.3, подтверждает, что выбор микросервисной архитектуры является оптимальным, научно обоснованным решением. Данный подход гарантирует достижение необходимых показателей отказоустойчивости, позволяет гибко имплементировать многоуровневые ролевые политики доступа и закладывает надежный фундамент для дальнейшего горизонтального масштабирования веб-системы при увеличении объемов обрабатываемых финансовых транзакций.

Большинство существующих облачных решений опираются на устаревшие методы парольной аутентификации, дополненные вторым фактором (2FA). Внедрение технологии беспарольного входа WebAuthn (Passkey) в существующие монолитные платформы (например, Nextcloud)

часто требует сложной настройки сторонних плагинов и не является нативным. Разработка собственной системы на современном стеке (FastAPI + Next.js) позволяет изначально заложить поддержку Passkey как основного метода входа, что полностью устраняет векторы атак, связанные с фишингом, перехватом паролей и их подбором (Brute-force), так как приватный ключ аутентификации никогда не покидает устройство пользователя.

Использование проприетарных облачных решений (SaaS) всегда сопряжено с риском наличия недокументированных возможностей («бэкдоров») или скрытой телеметрии. Даже Open Source решения (Nextcloud) из-за своей избыточной функциональности (календари, чаты, почта) имеют огромную кодовую базу, аудит которой затруднителен. Создание специализированного микросервисного приложения с минимально необходимым набором функций (загрузка, скачивание, шифрование) позволяет обеспечить полную прозрачность алгоритмов защиты и минимизировать поверхность атаки (Attack Surface Reduction).

Разработка собственного прототипа защищенного хранилища позволяет на практике исследовать эффективность применения современных криптографических библиотек и систем управления секретами в условиях, приближенных к реальным. Это дает возможность гибко экспериментировать с алгоритмами шифрования и схемами распределения ключей, что невозможно при использовании готовых коммерческих продуктов с закрытой архитектурой.

Таким образом, разработка собственной системы является обоснованной и необходимой для решения задачи обеспечения конфиденциальности данных в условиях не доверенной среды, устраняя недостатки существующих аналогов за счет архитектурной изоляции ключей и применения фишинго-устойчивой аутентификации.

1.5. Формирование требований к разрабатываемой веб-системе

Процесс инженерного проектирования любой высоконагруженной программной платформы, в особенности оперирующей в секторе финансовых

технологий, требует этапа строгой формализации требований. Специфика предметной области, сопряженная с управлением трансграничными переводами (как в фиатном, так и в криптовалютном эквивалентах, в частности направления FIAT_FIAT, CRYPTO_FIAT, FIAT_CRYPTO), диктует необходимость составления исчерпывающего перечня функциональных и нефункциональных (архитектурных, эксплуатационных) требований. Данный этап является критически важным, поскольку именно формализованные спецификации ложатся в основу выбора технологического стека, проектирования топологии баз данных и алгоритмов обеспечения информационной безопасности.

Функциональные требования описывают то, какие конкретные задачи и бизнес-сценарии должна реализовывать разрабатываемая веб-система. На основе проведенного анализа предметной области (разделы 1.1 и 1.2) был сформирован следующий базис функциональных требований:

1. Реализация многоуровневого конечного автомата (State Machine) управления сделками. Система обязана поддерживать параллельное ведение множественных статусных моделей в рамках единой сущности заявки. Это подразумевает независимое, но логически связанное отслеживание статусов жизненного цикла лида (DealStatusLead), взаимодействия с агентом (DealStatusAgent) и взаимодействия с конечным провайдером ликвидности — субагентом (DealStatusSubagent). Программная логика должна алгоритмически блокировать любые попытки несанкционированного изменения состояния: например, переход в статус подтверждения финансового объема (WORKING_WITH_VOLUME или RECEIVING_VOLUME) возможен исключительно после успешного завершения этапов верификации контрагента и согласования инвойсов.

2. Обеспечение гранулярного ролевого доступа (RBAC). Система должна предоставлять механизмы динамической конфигурации прав доступа на уровне отдельных атрибутов данных и операций.

Интерфейс и API должны быть спроектированы таким образом, чтобы пользователи с ролью «Менеджер» (MANAGER) или «Комплаенс-офицер» получали доступ исключительно к разрешенным им представлениям данных. Должна быть обеспечена возможность скрывания критичных финансовых метрик (например, комиссий субагентов) от неавторизованных ролей.

3. Управление документооборотом и файловыми хранилищами. Платформа должна поддерживать загрузку, валидацию, хранение и генерацию строго типизированных файлов и документов. В их число входят, но не ограничиваются ими: контракты от агентов (CONTRACT_FROM_AGENT_SUBAGENT), инвойсы от клиентов (INVOICE_FROM_CUSTOMER), ордера для субагентов (ORDER_SUBAGENT) и закрывающие акты (ACT_FOR_CUSTOMER). Загрузка документа определенного типа должна выступать триггером для перехода заявки на следующий этап бизнес-процесса.

4. Интеграция подсистемы коммуникаций в реальном времени. Для обеспечения оперативного разрешения спорных ситуаций (апелляций) и коммуникации между участниками (например, в статусе DEAL_STATUS_AGENT_APPEAL) требуется реализация встроенного модуля чатов. Данный модуль должен поддерживать различные контексты общения: внутренние служебные чаты (DEAL_CHAT_TYPE_INTERIOR), клиентские чаты (DEAL_CHAT_TYPE_CLIENT) и чаты апелляций (DEAL_CHAT_TYPE_APPEAL). Обмен сообщениями должен осуществляться мгновенно без необходимости перезагрузки веб-страниц.

Нефункциональные требования определяют качественные характеристики программного комплекса, его поведение в условиях пиковых

нагрузок, а также критерии отказоустойчивости и информационной безопасности:

1. Требования к производительности и пропускной способности. Архитектура веб-системы должна обеспечивать минимальное время отклика (Latency) для критических API-запросов (не более 200 миллисекунд для операций чтения и не более 500 миллисекунд для транзакционных операций с базами данных). Система обмена сообщениями должна обрабатывать тысячи одновременных WebSocket-соединений, обеспечивая доставку сообщений с задержкой не более 100 миллисекунд.

2. Требования к масштабируемости и развертыванию. В связи с прогнозируемым ростом объема обрабатываемых заявок, система должна быть спроектирована с учетом возможностей горизонтального масштабирования (Horizontal Scaling). Микросервисы должны быть контейнеризированы (Docker) и управляться посредством средств оркестрации (Docker Swarm). Обновление компонентов должно производиться без остановки обслуживания пользователей (Zero-downtime deployment) через конвейеры непрерывной интеграции (CI/CD).

3. Требования к сетевой инфраструктуре и защите от перегрузок (DDoS-защита). Учитывая финансовую направленность системы и высокие риски целевых кибератак, требуется реализация эшелонированной системы защиты. Внешний периметр должен защищаться программно-аппаратными комплексами глубокой инспекции трафика (в частности, интеграция Service Pipe и межсетевого экрана уровня приложений Solidwall, с последующим терминированием на реверс-прокси Nginx). На уровне самого приложения (Application Layer) требуется внедрение микросервиса API Gateway, задачей которого будет являться ограничение частоты запросов (Rate Limit) на основе алгоритма «Скользящего окна»

(Sliding Window), состояние которого будет распределенно храниться в высокоскоростном In-memory кластере.

4. Требования к консистентности и хранению данных. Система должна гарантировать транзакционную целостность финансовых и статусных данных в соответствии с требованиями ACID (Atomicity, Consistency, Isolation, Durability). Для реляционных данных предполагается использование кластера баз данных с балансировкой пулов соединений (Pgpool), а для потоковых данных чатов и событий логирования — высокопроизводительных нереляционных хранилищ или брокеров сообщений (RabbitMQ Cluster).

Сформулированный выше комплекс требований охватывает все аспекты будущего функционирования продукта. Описанные метрики производительности и жесткие критерии информационной безопасности не оставляют пространства для использования монолитных решений, окончательно подтверждая необходимость разработки распределенной микросервисной архитектуры с выделенными узлами для маршрутизации трафика и асинхронной обработки событий.

Выводы по первой главе

В первой главе выпускной квалификационной работы было проведено глубокое, системное исследование предметной области, охватывающей специфику управления жизненным циклом трансграничных финансовых заявок (в фиатных и криптовалютных эквивалентах). Выполненный теоретический и практический анализ позволил выявить фундаментальные проблемы отрасли: наличие длительных бюрократических процедур согласования, высокие риски операционных ошибок при ручном управлении статусами, а также критическую важность обеспечения конфиденциальности финансовых данных субагентов и агентов.

В ходе исследования была обоснована безальтернативная необходимость применения строгой ролевой модели управления доступом (Role-Based Access Control). Было продемонстрировано, что только сегментация обязанностей между менеджерами, комплаенс-офицерами, юристами и финансистами способна исключить возникновение конфликта интересов внутри платформы и обеспечить прозрачность движения капиталов.

Проведенный сравнительный анализ существующих на рынке программных продуктов класса CRM (как универсальных SaaS-платформ, так и Enterprise-монолитов) выявил их концептуальную неспособность удовлетворить специфическим требованиям финансового процессинга. Установлено, что классическая плоская структура данных и линейные воронки продаж не способны поддерживать параллельное исполнение многомерных конечных автоматов, которые необходимы для ведения сложных сделок. Более того, внедрение модулей реального времени (встроенных чатов) в монолитную архитектуру ведет к неминуемому снижению общей отказоустойчивости программного комплекса.

На основании выявленных ограничений было аргументировано решение о проектировании и разработке собственной (Custom) веб-системы на базе микросервисной архитектуры (Microservices Architecture). Детализированы преимущества использования паттернов API Gateway для фильтрации трафика и реализации механизмов защиты от перегрузок, а также событийно-ориентированного подхода (Event-Driven Architecture) с использованием брокеров сообщений для асинхронного взаимодействия подсистем.

Логическим итогом проведенного анализа стало формирование

исчерпывающего перечня функциональных и нефункциональных требований. Сформулированные требования к алгоритмической блокировке несанкционированных действий (State Machine), эшелонированной защите (с использованием межсетевых экранов Solidwall, Nginx и микросервиса-шлюза), ограничению частоты запросов (Sliding Window Rate Limit), а также жесткие критерии к горизонтальному масштабированию и автоматизации развертывания (CI/CD) создают надежный научный и инженерный фундамент.

Сформированная теоретическая и аналитическая база является полностью достаточной для перехода ко второму этапу исследования — непосредственному архитектурному проектированию слоев веб-системы, разработке логических моделей баз данных и алгоритмизации бизнес-процессов, что будет подробно освещено в последующей главе работы.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И БИЗНЕС-ЛОГИКИ СЕРВИСА УПРАВЛЕНИЯ ЗАЯВКАМИ

2.1. Разработка общей архитектуры программного комплекса и топологии сети

Проектирование архитектуры высоконагруженной веб-системы для финансового сектора требует строгого соблюдения принципов эшелонированной защиты (Defense in Depth), высокой доступности (High Availability) и горизонтальной масштабируемости. Исходя из проанализированных в первой главе ограничений монолитных систем, базовым архитектурным паттерном для разрабатываемого решения была выбрана микросервисная парадигма (Microservices Architecture). Данный подход предполагает логическую и физическую декомпозицию приложения на набор независимых узлов, взаимодействие между которыми осуществляется по легковесным сетевым протоколам.

При разработке общей топологии сети фундаментальной задачей являлось обеспечение надежной изоляции внутренних бизнес-процессов от внешних угроз. Для реализации данной задачи сетевая инфраструктура была разделена на несколько изолированных контуров (слоев), каждый из которых выполняет строго определенную функцию фильтрации, маршрутизации или обработки данных. Информационный поток (жизненный цикл HTTP-запроса), инициируемый клиентским приложением, проходит последовательную многоступенчатую обработку перед тем, как достичь ядра бизнес-логики.

Первый уровень: Периметр безопасности и фильтрация трафика

Для защиты веб-системы от распределенных атак типа «отказ в обслуживании» (DDoS) на сетевом (L3) и транспортном (L4) уровнях модели OSI, первичной точкой входа является сервис защиты от DDoS-атак (Service Pipe). Данный узел осуществляет поведенческий анализ входящего трафика, отсекая объемные (volumetric) атаки, такие как UDP-флуд или SYN-флуд, тем самым предотвращая исчерпание пропускной способности магистральных каналов связи.

Очищенный от сетевого «мусора» трафик перенаправляется на уровень защиты приложений (L7 модели OSI), где развернут межсетевой экран уровня веб-приложений — Solidwall (Web Application Firewall, WAF). Использование Solidwall обусловлено необходимостью глубокой инспекции пакетов (Deep Packet Inspection). WAF анализирует тело каждого HTTP-запроса на предмет наличия сигнатур известных уязвимостей, таких как внедрение SQL-кода (SQL Injection), межсайтовый скриптинг (XSS), а также осуществляет блокировку запросов, нарушающих спецификации протокола HTTP. Таким образом, до внутренней инфраструктуры допускается исключительно легитимный, семантически корректный трафик.

Второй уровень: Реверс-проксирование и терминация SSL

За слоем WAF располагается балансировщик нагрузки и реверс-прокси сервер на базе Nginx. В рамках спроектированной топологии на данный узел возложены две критические задачи. Во-первых, Nginx осуществляет терминацию SSL/TLS-соединений (расшифровку HTTPS-трафика). Вынос криптографических операций по расшифровке трафика на отдельный высокопроизводительный узел позволяет существенно снизить вычислительную нагрузку на внутренние микросервисы. Во-вторых, Nginx выполняет первичную балансировку запросов (по алгоритму Round Robin или Least Connections) между доступными репликами внутреннего микросервисного шлюза, обеспечивая отказоустойчивость на случай падения одного из узлов.

Третий уровень: API Gateway (Микросервисный шлюз)

Центральным инфраструктурным элементом, связывающим внешнюю среду с внутренними сервисами, является API Gateway. Данный компонент спроектирован как самостоятельный микросервис. В его зону ответственности входит маршрутизация запросов к конкретным микросервисам (например, запросы к /api/v1/deals перенаправляются в сервис управления сделками), проверка аутентификационных данных (валидация JWT-токенов) и применение политик ограничения частоты запросов (Rate Limit). Для реализации распределенного хранения счетчиков запросов шлюз

имеет эксклюзивный доступ к высокоскоростному In-Memory кластеру Redis. Это предотвращает перегрузку системы даже в случае, если легитимный авторизованный пользователь попытается отправить аномальное количество запросов в секунду (например, в результате ошибки в клиентском коде или попытки парсинга данных).

Четвертый уровень: Бизнес-логика и хранение данных

На самом глубоком уровне изоляции, не имеющем прямого доступа из глобальной сети Интернет (в так называемой Private Subnet), развернуты микросервисы ядра системы (Core Backend). Именно здесь инкапсулирована логика управления конечными автоматами сделок, расчета комиссий, валидации юридических статусов компаний и генерации закрывающих документов.

Для обеспечения транзакционной целостности данных (ACID) спроектирована подсистема хранения на базе реляционной СУБД PostgreSQL. Учитывая специфику микросервисов, которые в процессе работы могут создавать сотни одновременных подключений к базе данных, прямая работа приложений с СУБД была признана нецелесообразной. С целью оптимизации использования оперативной памяти сервера баз данных и предотвращения исчерпания лимита соединений (Connection Exhaustion), в топологию был внедрен промежуточный балансировщик соединений — Pgpool (PostgreSQL Connection Pool). Pgpool осуществляет мультиплексирование клиентских сессий, кеширование запросов и балансировку транзакций между мастер-узлом и репликами базы данных.

Для наглядного представления спроектированной многоуровневой архитектуры и маршрутизации информационных потоков была разработана диаграмма развертывания (Deployment Diagram), представленная на рисунке 2.1.

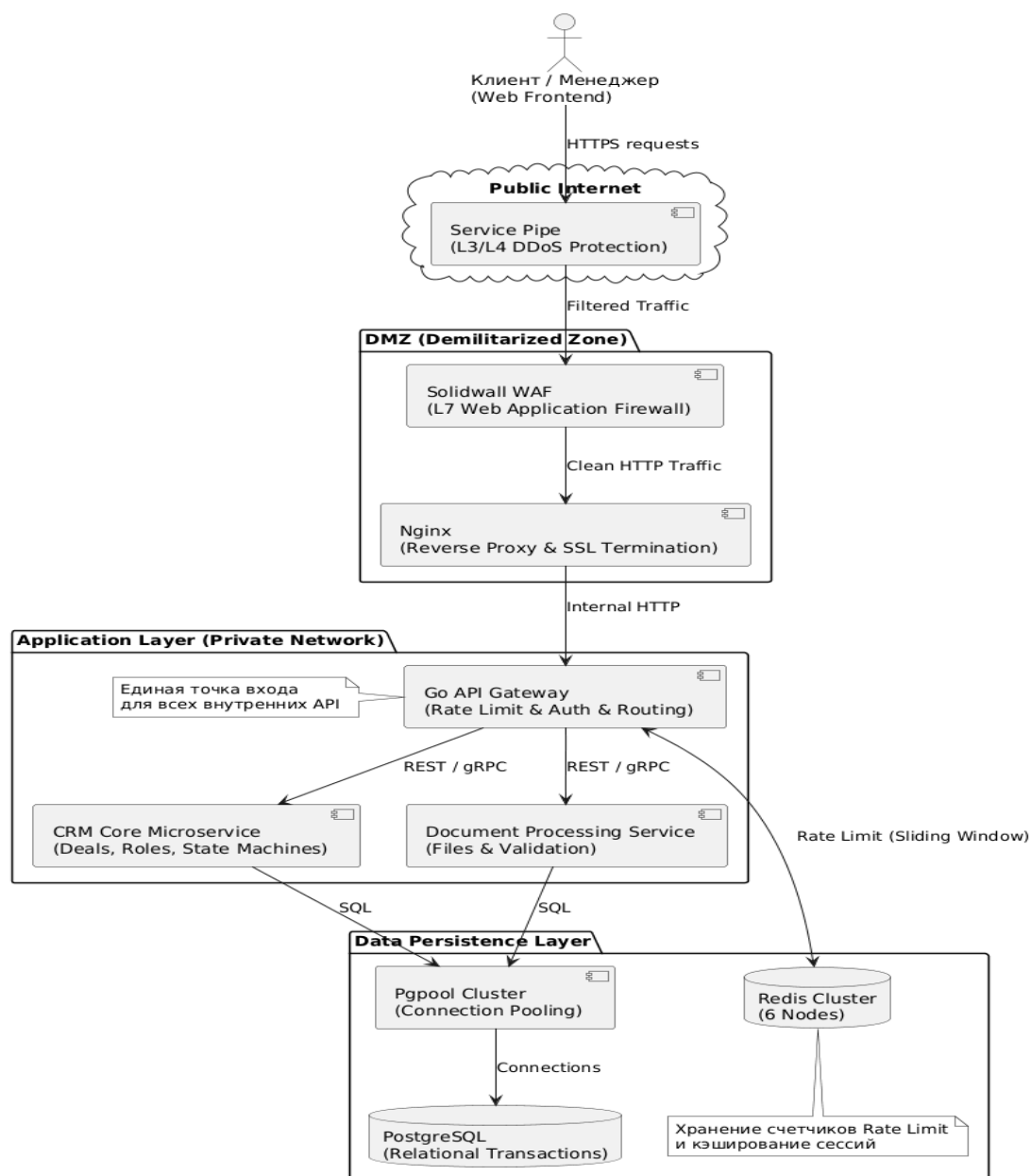


Рисунок 2.1 — Диаграмма развертывания и топологии сети программного комплекса

2.2. Обоснование выбора стека технологий реализации

Выбор технологического стека представляет собой один из наиболее ответственных и критичных этапов жизненного цикла проектирования программного обеспечения. В контексте разработки высоконагруженной финансовой веб-системы, оперирующей транзакционными данными и требующей неукоснительного соблюдения политик безопасности, технологический стек должен обеспечивать синергию производительности,

отказоустойчивости и скорости разработки. На основании сформулированных в первой главе функциональных и нефункциональных требований, а также спроектированной в разделе 2.1 топологии сети, был проведен системный анализ и отбор языков программирования, фреймворков и инфраструктурных решений для каждого слоя архитектуры.

Слой клиентского интерфейса (Frontend): React и Next.js

Для реализации пользовательского интерфейса (фронтенда) был выбран компонентный подход на базе библиотеки React. Выбор React обусловлен использованием механизма Virtual DOM, который обеспечивает высокую скорость рендеринга и обновления элементов интерфейса без необходимости полной перезагрузки веб-страницы. В условиях сложной CRM-системы, где интерфейс менеджера или комплаенс-офицера насыщен множеством динамических таблиц, графиков и статусов сделок, реактивность интерфейса является критическим требованием для обеспечения комфортной работы пользователя (User Experience).

В качестве надстройки над React был внедрен мета-фреймворк Next.js. Целесообразность его применения детерминирована несколькими факторами. Во-первых, Next.js предоставляет встроенные механизмы гибридного рендеринга: серверного (Server-Side Rendering, SSR) и статической генерации (Static Site Generation, SSG). Это позволяет безопасно скрывать часть бизнес-логики маршрутизации на сервере и минимизировать объем JavaScript-кода, загружаемого в браузер клиента. Во-вторых, строгая структура директорий и встроенная система маршрутизации (App Router) Next.js идеально подходят для реализации сложных ролевых панелей управления (дашбордов), где доступ к определенным страницам (например, странице аудита для роли Контролер) должен блокироваться еще на этапе серверного формирования ответа.

Слой маршрутизации и защиты (API Gateway): Язык Go

Как было определено ранее, микросервисный шлюз (API Gateway) выступает единой точкой входа для всех внутренних сервисов. На данный узел ложится колоссальная нагрузка по обработке каждого входящего HTTP-

запроса, валидации криптографических подписей JWT-токенов и реализации алгоритмов ограничения частоты обращений (Rate Limit). Для решения этих задач использование интерпретируемых языков программирования признано неэффективным из-за высоких накладных расходов на потребление оперативной памяти и управление потоками.

В качестве инструмента реализации микросервиса-шлюза был выбран компилируемый язык программирования Go (Golang). Архитектурным преимуществом Go является его встроенная модель параллелизма, базирующаяся на легковесных потоках исполнения — горутинах (Goroutines). Запуск тысяч горутин для обработки параллельных сетевых соединений требует минимальных затрат памяти (порядка нескольких килобайт на одну горутину), что позволяет шлюзу на базе Go обрабатывать десятки тысяч запросов в секунду (RPS) на стандартном аппаратном обеспечении. Кроме того, строгая статическая типизация и высокая скорость компиляции минимизируют риски возникновения ошибок во время исполнения (Runtime Errors) на критически важном пограничном узле сети.

Слой бизнес-логики (Core Backend): Python и FastAPI

Для разработки микросервисов, реализующих сложную предметную логику (управление конечными автоматами, проверку ролевых доступов, обработку файлов и расчет комиссионных вознаграждений), был выбран язык Python в связке с современным веб-фреймворком FastAPI.

Несмотря на то, что Python уступает Go в скорости математических вычислений, его применение в слое бизнес-логики абсолютно оправдано. Python обладает непревзойденной по своему объему экосистемой библиотек для работы с финансовыми вычислениями, криптографией и генерацией документов. Выбор фреймворка FastAPI обусловлен его архитектурными особенностями:

Нативная асинхронность. FastAPI построен на базе спецификации ASGI (Asynchronous Server Gateway Interface), что позволяет эффективно обрабатывать операции ввода-вывода (I/O-bound tasks), такие как запросы к базам данных или сторонним API, не блокируя основной поток выполнения.

Строгая валидация данных. Интеграция с библиотекой Pydantic позволяет декларативно описывать модели данных. Любой входящий запрос (например, полезная нагрузка для смены статуса на DEAL_STATUS_SUBAGENT_WORKING_WITH_VOLUME) автоматически проверяется на соответствие типам, заявленным в Enum-классах. При несовпадении типов фреймворк автоматически генерирует подробный ответ с кодом ошибки 422 (Unprocessable Entity), что исключает попадание некорректных данных в базу.

Автоматическая генерация документации. FastAPI "из коробки" генерирует интерактивную документацию в форматах Swagger UI и ReDoc на основе спецификации OpenAPI. Это критически важно для синхронизации работы команд бэкенд- и фронтенд-разработки в условиях микросервисной архитектуры.

Слой хранения данных (Data Persistence): PostgreSQL, Pgpool и Redis

Обеспечение целостности транзакционных данных (информация о пользователях, статусах сделок, балансах) требует использования надежной реляционной системы управления базами данных (СУБД). В качестве таковой была выбрана PostgreSQL. Данная СУБД полностью соответствует стандарту ACID, поддерживает сложные механизмы блокировок строк (Row-level locking) для предотвращения состояния гонки (Race conditions) при одновременном изменении статуса заявки разными пользователями, а также обладает встроенной поддержкой типа данных JSONB для гибкого хранения неструктурированных метаданных.

Для масштабирования подключений к PostgreSQL внедрен инструмент Pgpool. В микросервисной среде сотни контейнеров могут одновременно запрашивать доступ к базе данных, что быстро исчерпает лимит подключений (max_connections) самой PostgreSQL. Pgpool решает эту проблему за счет мультиплексирования пула соединений, удерживая открытые сокеты и переиспользуя их для новых запросов от микросервисов.

В качестве высокоскоростного хранилища данных в оперативной памяти (In-Memory Database) интегрирован кластер Redis из 6 узлов. Redis выполняет три ключевые функции: хранение временных окон и счетчиков для алгоритма Rate Limit (так как операции инкремента в Redis атомарны), кэширование часто запрашиваемых справочников (например, списков стран или типов валют) для снижения нагрузки на PostgreSQL, а также хранение пользовательских сессий. Кластерный режим обеспечивает шардирование данных и высокую доступность.

Слой асинхронного взаимодействия: RabbitMQ и Centrifugo

Для реализации событийно-ориентированного взаимодействия между микросервисами выбран надежный брокер сообщений RabbitMQ, работающий по протоколу AMQP. Его применение позволяет развязать (decouple) процессы: например, когда статус сделки меняется на IMPLEMENTED, микросервис CRM просто публикует событие в очередь RabbitMQ. Остальные микросервисы (отправка email, генерация отчетов) асинхронно вычитывают это событие и выполняют свою работу, не задерживая HTTP-ответ клиенту.

Для организации мгновенного обмена сообщениями в чатах (внутренних, клиентских, апелляционных) традиционный подход с удержанием WebSocket-соединений непосредственно в Python-приложении признан неэффективным (из-за особенностей работы интерпретатора Python и блокировок GIL). Для решения этой задачи в стек технологий внедрен кластер Centrifugo — специализированный сервер обмена сообщениями реального времени. Клиентское приложение на Next.js устанавливает постоянное WebSocket-соединение с Centrifugo. Когда бэкенд на Python должен отправить сообщение в чат, он делает легковесный HTTP-вызов к API Centrifugo (или публикует событие в Redis/RabbitMQ), а Centrifugo берет на себя ресурсоемкую задачу по широковещательной рассылке (Broadcasting) этого сообщения тысячам подключенных клиентов.

Для визуализации архитектурного соответствия выбранных технологий конкретным слоям программного комплекса была разработана логическая диаграмма стека технологий, представленная на рисунке 2.2.

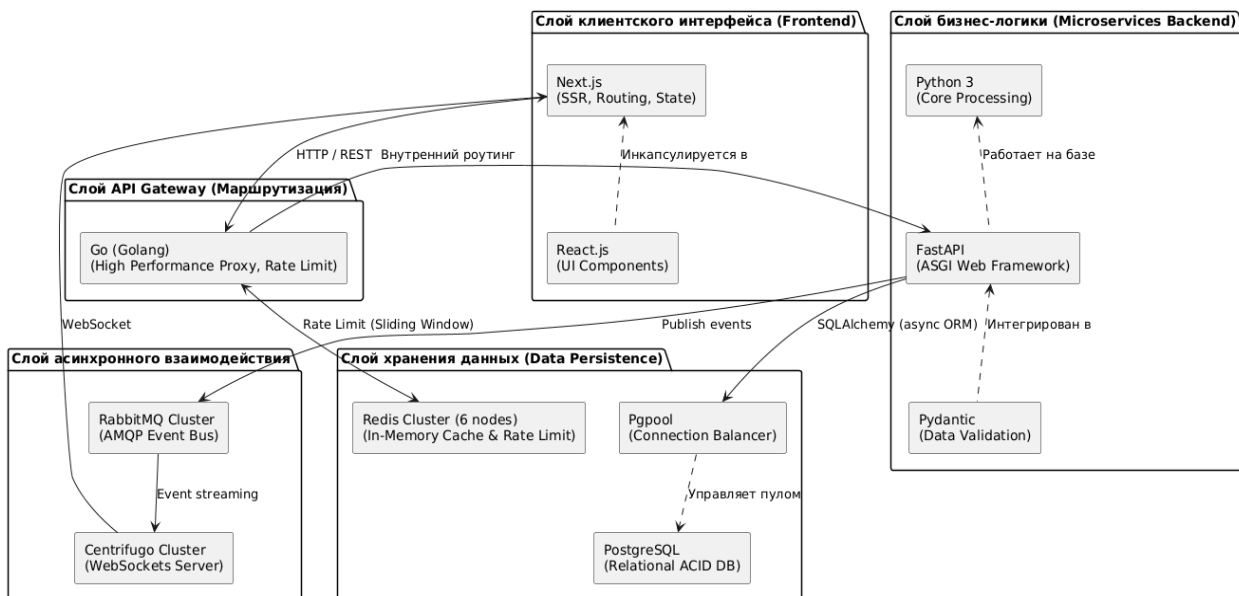


Рисунок 2.2 — Структурная схема технологического стека и взаимодействия компонентов

Проведенный анализ и представленная на рисунке 2.2 диаграмма наглядно демонстрируют, что выбранный технологический стек полностью покрывает все аспекты предметной области. Комбинация высокопроизводительного Go для шлюза, гибкого Python/FastAPI для сложной бизнес-логики, строгого PostgreSQL для транзакций и Centrifugo для интерактивных коммуникаций образует монолитный по своей надежности, но распределенный по своей природе программный фундамент, готовый к реализации бизнес-задач любой степени сложности.

2.3. Проектирование микросервиса API Gateway и подсистемы защиты от перегрузок

Проектирование микросервисного шлюза (API Gateway) представляет собой один из наиболее технологически сложных и ответственных этапов

разработки распределенной финансовой платформы. В условиях микросервисной архитектуры предоставление клиентским приложениям прямого доступа к внутренним бизнес-сервисам является категорически недопустимым (антипаттерн Direct Client-to-Microservice Communication). Прямое взаимодействие приводит к сильной связанности компонентов, усложняет рефакторинг, делает невозможным централизованный контроль безопасности и многократно увеличивает площадь потенциальной атаки (Attack Surface). Для решения данной архитектурной проблемы был спроектирован единый микросервис-шлюз на языке программирования Go, который выступает в роли фасадного паттерна (Facade) для всей внутренней инфраструктуры.

В рамках разрабатываемой веб-системы на микросервис API Gateway возложены три фундаментальные задачи: первичная аутентификация и проверка целостности токенов доступа, динамическая маршрутизация запросов к соответствующим доменам бизнес-логики, а также проактивная защита инфраструктуры от перегрузок (Rate Limiting).

2.3.1. Механизм аутентификации и валидации JWT-токенов

Каждый HTTP-запрос, поступающий от клиентского приложения (Next.js) на слой API Gateway, содержит заголовок авторизации с токеном стандарта JWT (JSON Web Token). Процесс валидации токена вынесен на уровень шлюза умышленно: это позволяет отсекал неавторизованные или скомпрометированные запросы на самых ранних этапах, не затрачивая вычислительные ресурсы базовых микросервисов (написанных на Python), отвечающих за транзакционную логику.

При получении запроса шлюз извлекает токен, проверяет его криптографическую подпись с использованием асимметричного или симметричного алгоритма шифрования (в зависимости от конфигурации удостоверяющего центра), а также верифицирует срок действия токена (параметр `exp` — `expiration time`). Если токен признан валидным, шлюз извлекает из полезной нагрузки (Payload) идентификатор пользователя и его

роль (например, MANAGER или FINANCIER), добавляет эти данные в виде безопасных внутренних HTTP-заголовков (например, X-User-ID, X-User-Role) и передает запрос далее по маршруту. В случае невалидности токена процесс немедленно прерывается, и клиенту возвращается стандартизированный ответ с HTTP-кодом 401 (Unauthorized).

2.3.2. Основные задачи разработки

Для достижения поставленной цели необходимо решить следующие задачи:

1. Спроектировать архитектуру информационной системы;
2. Определить состав ролей пользователей и разграничение доступа;
3. Разработать структуру каталога товаров с учетом предметной области;
4. Спроектировать модель данных для хранения информации о пользователях, товарах, заказах и отзывах;
5. Реализовать процессы регистрации, авторизации и управления профилями;
6. Обеспечить создание и публикацию карточек товаров продавцами;
7. Реализовать поиск, фильтрацию и просмотр каталога для покупателей;
8. Реализовать оформление и сопровождение заказов;
9. Обеспечить административное управление платформой;
10. Учесть требования масштабируемости, производительности и расширяемости.

Таким образом, постановка задач отражает как бизнес, так и технические аспекты итогового решения.

2.3.3. Обоснование и проектирование алгоритма ограничения частоты запросов (Rate Limit)

Финансовые системы являются приоритетной целью для злоумышленников. Помимо объемных DDoS-атак, которые отсекаются на уровне L3/L4 (Service Pipe) и L7 (Solidwall WAF), существует угроза

логических атак (Application-layer DDoS) и попыток автоматизированного перебора данных (Brute Force, API Scraping). Например, злоумышленник может попытаться массово генерировать запросы к эндпоинту смены статуса сделки или к сервису выгрузки инвойсов. Для нейтрализации подобных угроз на уровне API Gateway необходимо внедрение механизма ограничения частоты запросов — Rate Limit.

В процессе проектирования подсистемы защиты был проведен сравнительный анализ четырех классических алгоритмов ограничения трафика. Результаты анализа представлены в таблице 2.1.

Наименование алгоритма	Принцип работы	Преимущества	Недостатки
Fixed Window (Фиксированное окно)	Временная шкала делится на жесткие интервалы (например, с 12:00 до 12:01). Счетчик сбрасывается в начале каждого окна.	Минимальное потребление памяти, простота алгоритмической реализации.	Эффект "граничного всплеска": злоумышленник может отправить двойной лимит запросов на границе двух окон (в 12:00:59 и 12:01:00).
Leaky Bucket (Дырявое ведро)	Запросы помещаются в очередь (ведро) и обрабатываются с постоянной скоростью. Излишки отбрасываются.	Сглаживание трафика, строгая гарантия пропускной способности для бэкенда.	Невозможность обработки кратковременных легитимных всплесков (Bursts). Медленный ответ для пользователя при заполнении
Token Bucket (Маркерная корзина)	В корзину с заданной скоростью добавляются маркеры. Каждый запрос забирает маркер. Нет маркера — отказ.	Позволяет обрабатывать легитимные кратковременные всплески активности.	Требует фоновых процессов для пополнения корзины, сложнее в точной настройке для микро-интервалов.

Sliding Window Log (Скольльзящее окно)	С о х р а н я е т временную метку (timestamp) каждого запроса. Окно "скользит" вместе с текущим временем.	А б с о л ю т н а я т о ч н о с т ь о г р а н и ч е н и я . Полное отсутствие э ф ф е к т а " г р а н и ч н о г о всплеска".	В ы с о к о е п о т р е б л е н и е оперативной памяти и вычислительных ресурсов при большом потоке (O(N) для хранения
--	--	--	--

Таблица 2.1 — Сравнительный анализ алгоритмов ограничения частоты запросов (Rate Limit)

На основании данных, представленных в таблице 2.1, классический алгоритм Fixed Window был отвергнут из-за уязвимости на границах временных отрезков, а алгоритм Leaky Bucket — из-за несовместимости с требованиями к отзывчивости пользовательского интерфейса (так как он искусственно задерживает обработку запросов).

Для разрабатываемой системы был спроектирован гибридный подход - Sliding Window (Скольльзящее окно с использованием логов времени), оптимизированный для работы в оперативной памяти. Учитывая необходимость высокой скорости вычислений и распределенности (так как API Gateway может быть развернут в нескольких репликах), состояние счетчиков решено хранить в кластере Redis.

Для обеспечения атомарности операций (исключения состояния гонки или Race Conditions, когда тысячи конкурентных горутин одновременно пытаются обновить один и тот же счетчик) алгоритм реализован в виде скрипта на языке Lua, который выполняется непосредственно внутри движка Redis.

Использование команды ZSET (упорядоченного множества) в Redis позволяет хранить временные метки (timestamps) в качестве весов (scores). В результате вызов ZREMRANGEBYSCORE за логарифмическое время $O(\log(N)+M)$ удаляет все события, которые произошли до начала текущего окна clear_before. Если скрипт возвращает значение 0, микросервис на языке Go прерывает цепочку обработки и немедленно отправляет клиенту HTTP-ответ со статус-кодом 429 Too Many Requests. Дополнительно в заголовки

ответа включается параметр `Retry-After`, информирующий клиентское приложение (`Next.js`) о том, через сколько секунд возможна повторная отправка запроса.

2.3.4. Маршрутизация и агрегация данных

После успешного прохождения барьеров аутентификации и ограничения частоты обращений, запрос передается в модуль динамической маршрутизации (`Reverse Proxy module`). Данный модуль на основе конфигурационных файлов или данных из системы `Service Discovery` (в случае динамического масштабирования) определяет целевой микросервис. Например, запросы с префиксом `/api/deals/*` транслируются к `FastAPI` приложению (`CRM Core`), а запросы `/api/chat/*` инициируют процесс выдачи токенов для кластера `Centrifugo`.

Выводы по второй главе

Во второй главе было выполнено комплексное проектирование архитектуры и инфраструктуры веб-системы управления финансовыми заявками. На основе требований сформирован целостный инженерный фундамент: определены принципы маршрутизации, обработки данных и защиты системы на всех уровнях.

Разработана сетевая архитектура с многоуровневой защитой (`Defense in Depth`): фильтрация трафика на уровнях `L3/L4`, применение `WAF` и использование `Nginx` как реверс-прокси. Ключевые сервисы и базы данных размещены в изолированном приватном контуре, что соответствует требованиям безопасности финансовых систем.

Обоснован выбор технологического стека: `Go` для `API Gateway` (высокая производительность), `Python` и `FastAPI` для бизнес-логики (гибкость и валидация данных), `PostgreSQL` для хранения данных, `Redis` для кэширования и `rate limiting`, `RabbitMQ` для асинхронного взаимодействия и `Centrifugo` для работы с `WebSocket`. Также реализована концепция

централизованного шлюза с проверкой JWT и алгоритмом ограничения запросов Sliding Window.

Дополнительно спроектирована модель конечного автомата для управления статусами заявок с учетом ролей (RBAC) и сложных сценариев обработки. В результате сформирован полноценный архитектурный базис, позволяющий перейти к этапу реализации, тестирования и развертывания системы.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1. Реализация серверной части бизнес-логики и REST API

интерфейсов

Программная реализация основного микросервиса управления заявками (CRM Core Service) выполнена на языке программирования Python с применением асинхронного веб-фреймворка FastAPI. Выбор данного инструментария позволил обеспечить высокую скорость обработки запросов ввода-вывода (I/O) за счет использования стандарта ASGI и драйверов асинхронного доступа к базам данных (таких как asyncpg для PostgreSQL).

Для обеспечения строгой типизации данных на уровне API применена библиотека Pydantic. Данный подход гарантирует, что любая полезная нагрузка (Payload), поступающая от клиентского приложения или микросервиса-шлюза, автоматически валидируется до момента передачи в функции бизнес-логики. В качестве основы для валидации состояний используются классы перечислений (Enum), спроектированные в рамках моделирования конечного автомата.

В листинге 3.1 представлен фрагмент программного кода, демонстрирующий структуру Pydantic-модели для обновления статуса агента и проверку принадлежности нового статуса к разрешенному перечислению.

```
from enum import Enum
from pydantic import BaseModel, Field

class DealStatusAgent(str, Enum):
    CHECKING_THE_INVOICE = "DEAL_STATUS_AGENT_CHECKING_THE_INVOICE"
    WORKING_WITH_THE_ORDER = "DEAL_STATUS_AGENT_WORKING_WITH_THE_ORDER"
    RECEIVING_VOLUME = "DEAL_STATUS_AGENT_RECEIVING_VOLUME"
    IMPLEMENTED = "DEAL_STATUS_AGENT_IMPLEMENTED"
    APPEAL = "DEAL_STATUS_AGENT_APPEAL"

class DealStatusUpdateRequest(BaseModel):
    deal_id: str = Field(..., description="Уникальный идентификатор сделки")
    new_status: DealStatusAgent = Field(..., description="Целевой статус")
    comment: str | None = Field(default=None, max_length=1000)

    class Config:
        schema_extra = {
            "example": {
                "deal_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
                "new_status": "DEAL_STATUS_AGENT_RECEIVING_VOLUME"
            }
        }
```

Листинг 3.1 — DTO-модели валидации данных на базе Pydantic

Реализация эндпоинтов (контроллеров) REST API выполнена с использованием механизма внедрения зависимостей (Dependency Injection), предоставляемого FastAPI. Это позволило элегантно интегрировать логику проверки прав доступа (RBAC). Заголовки, которые были предварительно проверены и добавлены микросервисом шлюза (API Gateway), извлекаются в функции-зависимости `get_current_user`, после чего передаются непосредственно в обработчик маршрута.

Ключевой задачей при реализации серверной логики стала программная инкапсуляция правил конечного автомата (FSM). При попытке изменения статуса сделки микросервис выполняет запрос к реляционной базе данных через асинхронный драйвер `asynpg`. Производится проверка текущего статуса и сопоставление запрашиваемого перехода с матрицей доступа. Переход осуществляется автоматически после соблюдения необходимых действий для его выполнения.

Взаимодействие с реляционной базой данных PostgreSQL организовано через кластер балансировщика `Pgpool`. Для предотвращения исчерпания сетевых соединений настройки пула SQLAlchemy (параметры `pool_size` и `max_overflow`) синхронизированы с лимитами конфигурации `Pgpool`. Таким образом, реализация серверной части полностью инкапсулирует бизнес-логику CRM-системы, гарантируя надежность транзакций и абсолютную изоляцию данных на программном уровне.

3.2. Реализация шлюза запросов и алгоритма Rate Limit

Программная реализация единого микросервисного шлюза (API Gateway) выполнена с использованием компилируемого языка программирования Go (Golang). Выбор данного инструментария для пограничного узла (Edge Node) системы обусловлен необходимостью обработки огромного массива параллельных сетевых соединений без существенного увеличения потребления оперативной памяти. Благодаря

встроенной модели конкурентности, основанной на легковесных потоках (горутинах), разработанный шлюз способен эффективно мультиплексировать входящий трафик, маршрутизируя его к соответствующим микросервисам ядра.

Основная парадигма, заложенная при реализации шлюза на Go, заключается в вынесении всех сквозных инфраструктурных задач (Cross-cutting concerns) за пределы слоя бизнес-логики. Внутренние микросервисы, написанные на Python, оперируют такими инструментами как драйвер `asynсrg` для высокоскоростного асинхронного доступа к реляционной базе данных, `dishka` для обеспечения инверсии управления и внедрения зависимостей (Dependency Injection), а также фреймворк `faststream` для публикации и обработки событий в шине сообщений `RabbitMQ`. Использование микросервисного шлюза позволяет полностью освободить эти узкоспециализированные компоненты от необходимости самостоятельно выполнять предварительную фильтрацию входящих HTTP-запросов и проверку криптографических подписей.

Процесс обработки каждого входящего запроса внутри шлюза разделен на несколько последовательных фаз. На первой фазе осуществляется терминация пользовательского запроса и проверка наличия заголовка авторизации (Authorization: Bearer). Микросервис на Go производит валидацию криптографической подписи JWT-токена с использованием публичного ключа, проверяет срок его действия и извлекает полезную нагрузку (Payload). Извлеченные данные, такие как уникальный идентификатор пользователя и его системная роль (например, *MANAGER* или *FINANCIER*), интегрируются в виде безопасных внутренних заголовков (например, X-User-ID, X-User-Role). Оригинальный токен при этом может быть удален из запроса. Этот механизм гарантирует, что внутренние компоненты системы, инициализируемые через провайдеры `dishka`, получают гарантированно достоверные данные о субъекте запроса.

Второй, наиболее вычислительно сложной фазой является реализация проактивной защиты инфраструктуры от перегрузок — механизма Rate Limit.

В соответствии со спроектированной архитектурой, на уровне шлюза реализован алгоритм «Скользящего окна» (Sliding Window). Состояние счетчиков запросов распределенно хранится в кластере Redis. Если для внутренних Python-сервисов взаимодействие с хранилищем осуществляется через библиотеку redis-py, то на стороне шлюза используется нативный Go-клиент для Redis, обеспечивающий пул постоянных TCP-соединений с кластером.

Для обеспечения абсолютной атомарности операций и исключения состояния гонки (Race Condition) при одновременной обработке запросов от одного пользователя, алгоритм скользящего окна инкапсулирован в виде Lua-скрипта. Данный скрипт кэшируется на серверах Redis и выполняется непосредственно внутри базы данных как единая транзакция. Микросервис на Go передает скрипту уникальный идентификатор ключа (сформированный из IP-адреса клиента или его ID), текущую временную метку с точностью до миллисекунд и параметры лимита. Если Redis возвращает сигнал о превышении установленного порога, маршрутизация запроса немедленно прерывается. В этом случае шлюз формирует и отправляет клиентскому приложению ответ с HTTP-кодом 429 (Too Many Requests) и заголовком Retry-After, информируя интерфейс о необходимости временной приостановки отправки данных.

Для наглядной демонстрации описанной логики взаимодействия компонентов при обработке сетевого запроса была разработана диаграмма последовательности (Sequence Diagram), представленная на рисунке 3.1.

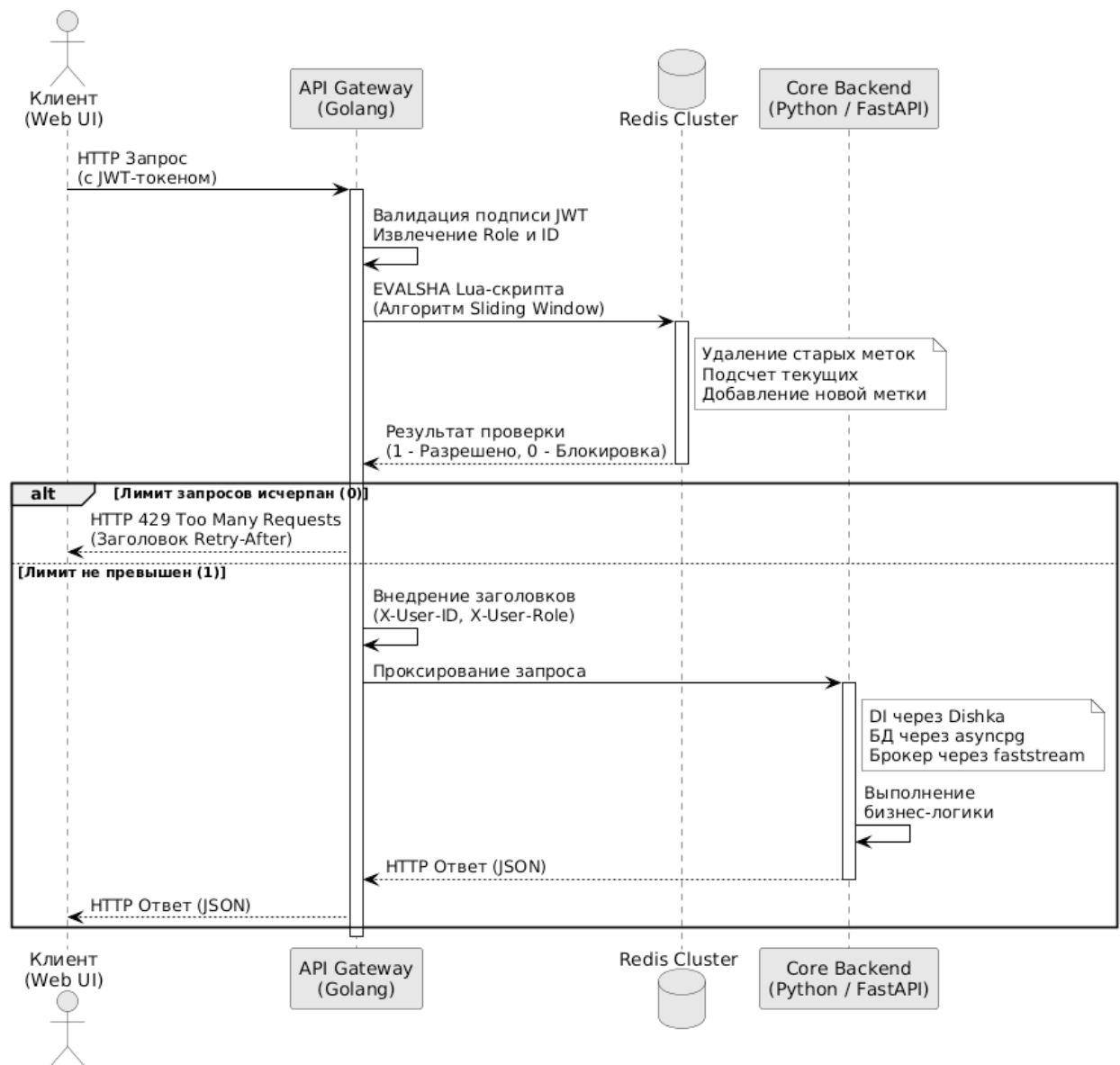


Рисунок 3.1 — Диаграмма последовательности обработки запроса через API Gateway

Третьей фазой является непосредственное реверс-проксирование (Reverse Proxying). Если запрос признан легитимным и не нарушает установленные скоростные лимиты, шлюз на базе Go устанавливает соединение с целевым микросервисом. Система динамической маршрутизации анализирует URL-путь запроса: например, все маршруты, начинающиеся с `/api/v1/deals`, прозрачно проксируются к ядру CRM-системы. Маршруты, связанные с инициализацией чатов (получение токенов для защищенного соединения), перенаправляются к соответствующему модулю,

который впоследствии обеспечивает выдачу клиентскому приложению токенов доступа для установления прямого WebSocket-соединения с кластером `centrifugo`.

Таким образом, реализованный микросервис шлюза выполняет функцию мощного, интеллектуального брандмауэра приложения. Написание данного компонента на языке Go позволило минимизировать время задержки (Latency), вносимое шлюзом в процесс обработки запроса, до единиц миллисекунд. Строгая алгоритмическая фильтрация трафика надежно изолирует транзакционное ядро системы на Python от любых аномальных всплесков нагрузки, гарантируя бесперебойное функционирование платформы в условиях промышленной эксплуатации.

3.4. Разработка клиентского веб-интерфейса и ролевого рендеринга

Разработка клиентской части (фронтенда) веб-системы представляет собой критически важный этап, определяющий качество взаимодействия конечных пользователей с программным комплексом. Ввиду высокой сложности предметной области, характеризующейся наличием множества параллельных статусных моделей и строгой иерархией доступа, к пользовательскому интерфейсу предъявляются повышенные требования в части реактивности, информационной безопасности и адаптивности. Для реализации клиентского веб-интерфейса был применен компонентный подход на базе библиотеки React в интеграции с мета-фреймворком Next.js.

Основополагающим принципом, заложенным в архитектуру фронтенд-приложения, является концепция динамического ролевого рендеринга (Role-Based Rendering). Несмотря на то, что фундаментальная защита данных и блокировка несанкционированных действий (RBAC) гарантированно обеспечиваются на стороне серверной бизнес-логики и микросервиса шлюза, клиентское приложение обязано предоставлять когерентный пользовательский опыт (User Experience). Это означает, что интерфейс должен превентивно скрывать элементы управления, недоступные для

текущей роли пользователя, исключая визуальное перегружение и предотвращая попытки вызова запрещенных API-методов.

Для решения этой задачи в архитектуру React-приложения внедрен глобальный контекст управления состоянием, в котором безопасно инкапсулируется профиль авторизованного пользователя, включая его уникальный идентификатор и системную роль. При отрисовке комплексных компонентов, таких как «Карточка сделки» (Deal Card), применяется паттерн компонентов высшего порядка (Higher-Order Components, HOC) и специализированные пользовательские хуки (Custom Hooks). Данные механизмы в реальном времени анализируют многомерный конечный автомат, получаемый от бэкенда в виде JSON-объекта, и принимают решение о рендеринге конкретных блоков.

Например, если сделка находится в статусе *DEAL_STATUS_AGENT_RECEIVING_VOLUME*, компонент анализирует права пользователя. Для роли «Финансист» отображается интерактивный блок с кнопками подтверждения получения средств и полями ввода фактических сумм. Для роли «Менеджер» этот же блок рендерится в режиме «Только для чтения» (Read-only), отображая лишь информационное уведомление о том, что сделка находится на этапе финансовой сверки. Аналогичный подход применяется к комментариям и вложениям: служебные заметки типа *DEAL_COMMENT_TYPE_AGREEMENT_SUBAGENT* алгоритмически исключаются из дерева DOM (Document Object Model), если интерфейс загружен под учетной записью клиента.

Для формализации процесса условной отрисовки пользовательского интерфейса в зависимости от контекста роли и текущего статуса бизнес-процесса разработана диаграмма деятельности (Activity Diagram), представленная на рисунке 3.2.

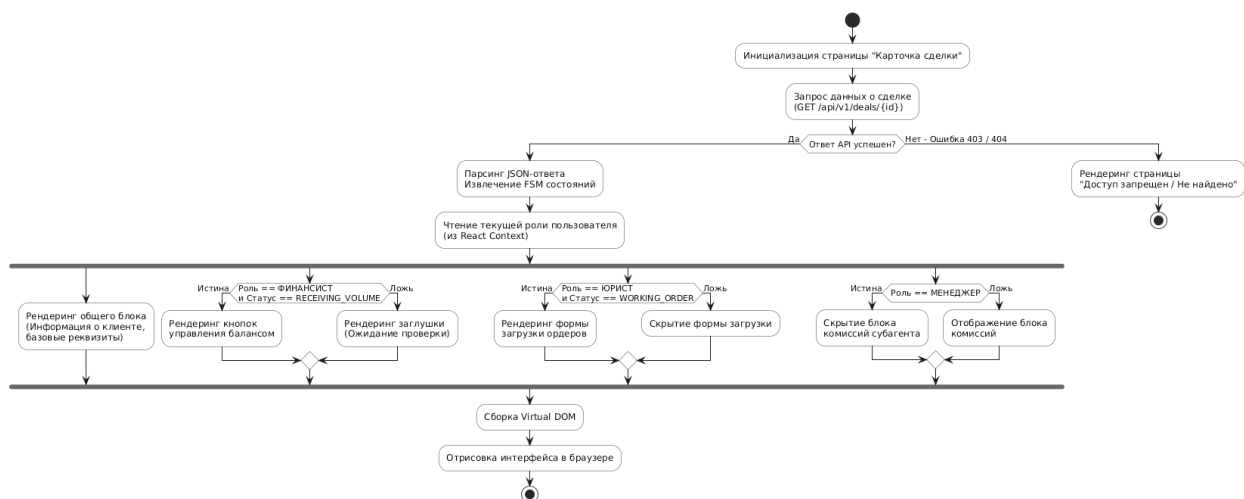


Рисунок 3.2 — Диаграмма логики ролевого рендеринга компонента "Карточка сделки"

Важнейшим аспектом реализации клиентской части стала разработка подсистемы управления документами. Процесс обработки трансграничных финансовых заявок сопровождается интенсивным обменом юридически значимыми файлами (свыше 25 типизированных форматов, включая *FILE_TYPE_INVOICE_FROM_CUSTOMER*, *FILE_TYPE_ACT_FOR_CUSTOMER* и др.). Во фронтенд-приложении реализован специализированный модуль загрузки, который перед отправкой файла на сервер осуществляет первичную валидацию его размера, формата и типа. Загрузка файла сопровождается динамическим индикатором прогресса (Progress Bar), а успешное сохранение документа выступает триггером для автоматического обновления статуса сделки в интерфейсе без необходимости ручной перезагрузки страницы пользователем.

Для обеспечения мгновенной обратной связи и реализации интерактивных функций (таких как встроенные чаты и всплывающие системные уведомления) клиентское приложение интегрировано с кластером Centrifugo. Традиционный опрос сервера (Long Polling) признан неэффективным ввиду высокой задержки и избыточной нагрузки на сетевую инфраструктуру. Вместо этого в приложении на Next.js реализован модуль, который при успешной авторизации запрашивает у бэкенда уникальный токен

доступа для подсистемы реального времени. С использованием данного токена фронтенд устанавливает постоянное, защищенное WebSocket-соединение с узлом Centrifugo.

Когда на стороне сервера происходит изменение состояния (например, комплаенс-офицер одобряет документы компании), сервер публикует событие в брокер сообщений. Centrifugo мгновенно транслирует это событие в открытый WebSocket-канал. Модуль клиентского приложения, подписанный на данный канал, перехватывает сообщение и инициирует локальное обновление состояния (State Update) в React. Это приводит к плавной анимации изменения статуса на экране менеджера в режиме реального времени, что минимизирует информационный разрыв между участниками процесса и значительно повышает общую эффективность работы с CRM-платформой.

Ключевым архитектурным принципом фронтенд-приложения является концепция динамического ролевого рендеринга (Role-Based Rendering). Несмотря на то, что контроль доступа и валидация прав пользователей реализованы на стороне серверной бизнес-логики, клиентское приложение выполняет дополнительную функцию — формирование корректного пользовательского опыта. Это достигается за счет скрывания или модификации элементов интерфейса в зависимости от роли пользователя, что снижает когнитивную нагрузку и исключает возможность взаимодействия с недоступными функциями.

На рисунке 3.3 представлен основной интерфейс системы, реализованный в виде канбан-доски, отражающей текущие этапы обработки сделок.

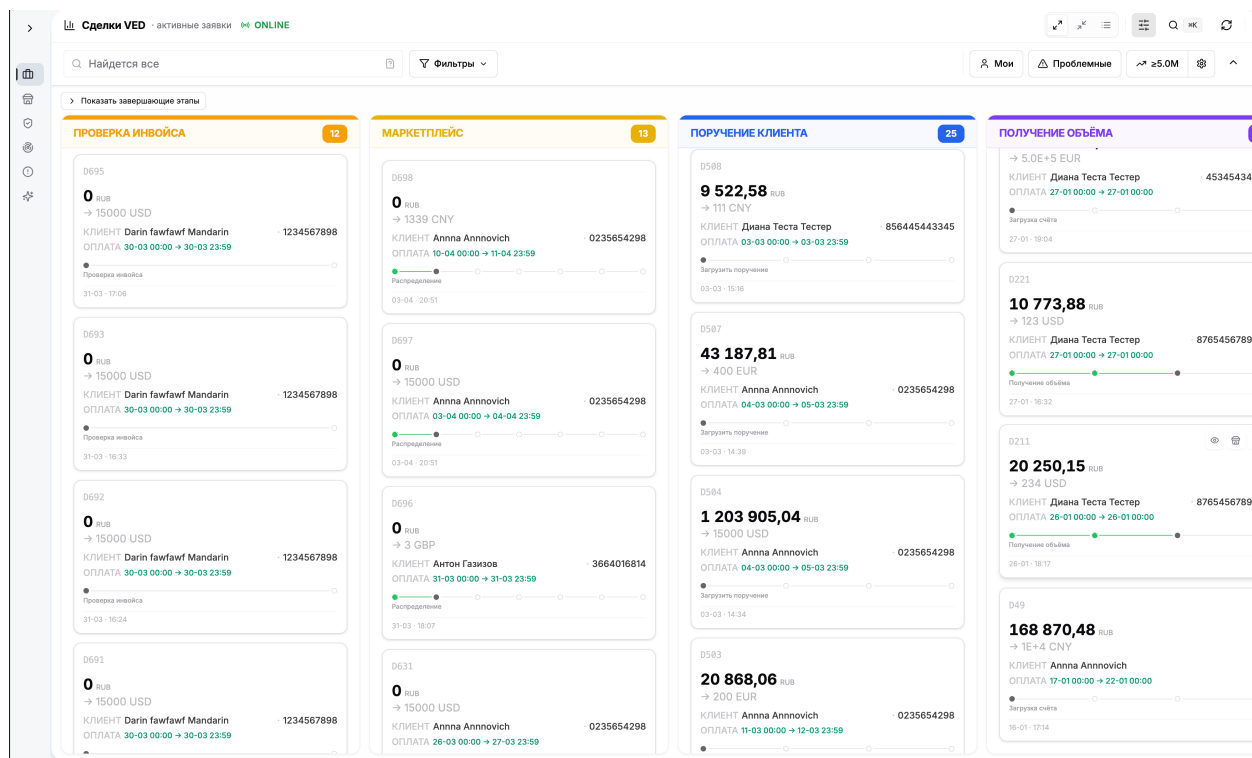


Рисунок 3.3 — Интерфейс отображения сделок в виде канбан-доски

Каждая колонка соответствует отдельному этапу бизнес-процесса (например, «Проверка инвойса», «Маркетплейс», «Поручение клиента»), а карточки внутри колонок представляют конкретные сделки. Такой подход обеспечивает наглядное отображение текущего состояния системы и позволяет пользователю быстро ориентироваться в процессе.

Детализированное представление сделки реализовано в виде отдельного интерфейса — «Карточки сделки», пример которого приведен на рисунке 3.4.

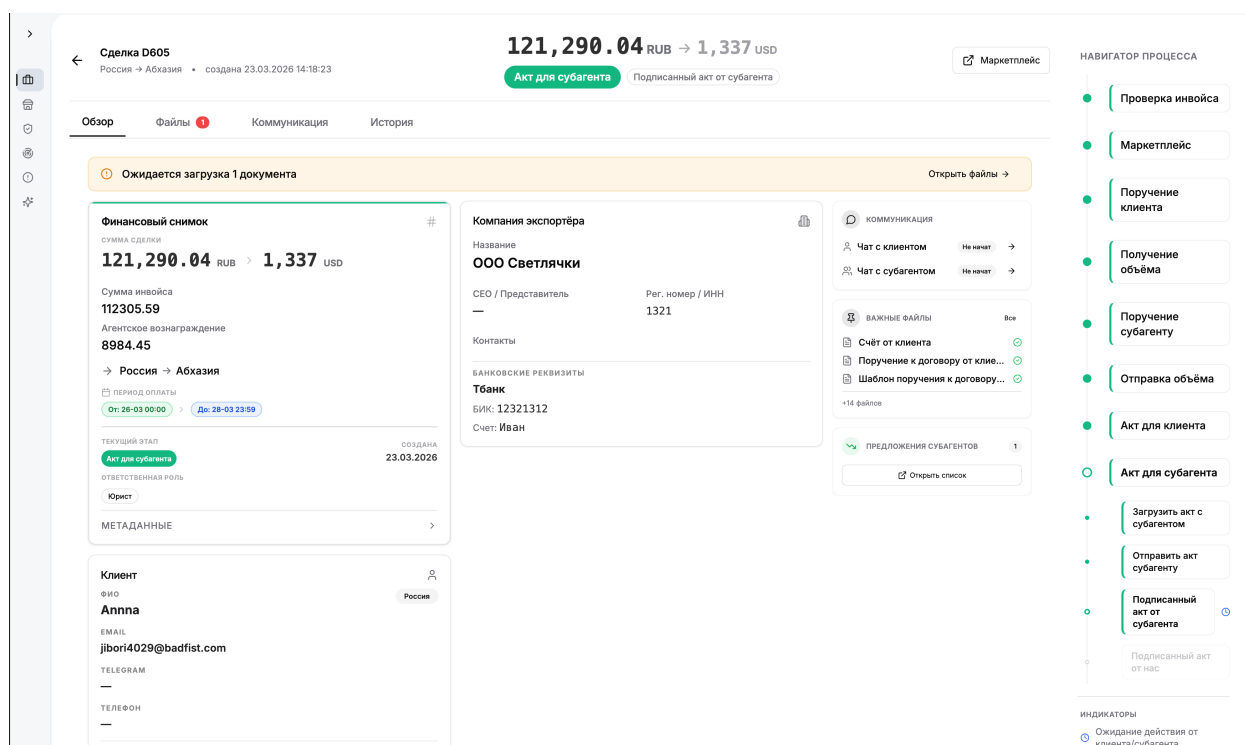


Рисунок 3.4 — Интерфейс карточки сделки с детализированной информацией и навигатором процесса

Данный интерфейс агрегирует ключевую информацию по сделке: финансовые показатели, данные клиента и контрагента, список документов, коммуникации и текущий статус выполнения. В правой части экрана реализован навигатор процесса, визуализирующий последовательность этапов обработки заявки.

3.4. Тестирование программного комплекса и оценка отказоустойчивости

Заключительным этапом разработки распределенной веб-системы управления финансовыми заявками стало проведение комплексного тестирования. Ввиду микросервисной архитектуры и высоких требований к финансовой безопасности, процесс верификации программного обеспечения был разделен на три основных направления: функциональное (модульное и интеграционное) тестирование бизнес-логики, нагрузочное тестирование микросервисного шлюза и оценка общей отказоустойчивости инфраструктуры.

3.4.1. Функциональное и интеграционное тестирование

Для верификации корректности работы ядра бизнес-логики (Core Backend), реализованного на языке Python, применялся фреймворк pytest. Основной фокус функционального тестирования был направлен на проверку конечного автомата (FSM) и системы управления доступом на основе ролей (RBAC). В ходе автоматизированных тестов эмулировались различные сценарии взаимодействия пользователей с системой.

Было подтверждено, что попытка пользователя с ролью «Менеджер» инициировать API-запрос на изменение статуса сделки на DEAL_STATUS_AGENT_RECEIVING_VOLUME корректно блокируется с возвратом HTTP-кода 403 (Forbidden). Аналогично, попытки нарушения хронологии конечного автомата (например, переход из статуса INITIAL сразу в IMPLEMENTED, минуя этапы загрузки инвойсов и согласования) успешно пресекались на уровне валидаторов Pydantic с возвратом ошибки 422 (Unprocessable Entity).

Интеграционное тестирование охватывало проверку межсервисного взаимодействия. В частности, тестировалась корректность работы механизма внедрения зависимостей (DI) на базе библиотеки dishka при инициализации пулов соединений asyncpg с базой данных PostgreSQL. Отдельным этапом проверялась асинхронная шина событий: с использованием фреймворка faststream эмулировалась публикация событий изменения статусов в RabbitMQ, после чего фиксировалась успешная доставка триггеров к модулю интеграции с кластером centrifugo для последующей рассылки WebSocket-уведомлений на клиентские устройства.

3.4.2. Нагрузочное тестирование и верификация алгоритма Rate Limit

Для оценки производительности и проверки работоспособности подсистемы защиты от перегрузок было проведено нагрузочное тестирование микросервисного шлюза (API Gateway), написанного на языке Go. В качестве инструмента генерации трафика использовалась утилита wrk (для генерации

высокоинтенсивных синтетических HTTP-запросов) и фреймворк Locust (для эмуляции поведения реальных пользователей).

Целью первого сценария нагрузочного тестирования являлась проверка алгоритма Sliding Window, реализованного через Lua-скрипты в кластере Redis. Для тестового эндпоинта был установлен лимит: 100 запросов в секунду (RPS) с одного IP-адреса. Генератор трафика инициировал отправку 500 запросов в секунду. Результаты мониторинга показали, что шлюз на Go успешно пропустил ровно 100 запросов, направив их к внутренним сервисам через Nginx, а для оставшихся 400 запросов за доли миллисекунды вернул ответ с HTTP-кодом 429 (Too Many Requests) и заголовком Retry-After. При этом потребление оперативной памяти микросервисом шлюза оставалось стабильным и не превышало 50 МБ, что подтверждает эффективность использования горутин для обработки сетевого ввода-вывода.

Во втором сценарии тестировалась общая пропускная способность системы при легитимном (распределенном) трафике. Сводные результаты нагрузочного тестирования основных узлов представлены в таблице 3.1.

Тестируемый компонент	Характер нагрузки	Пиковое значение RPS	Среднее время отклика (Latency)	Процент ошибок (Error Rate)
Go API Gateway (Только валидация JWT)	Распределенный GET-трафик	15 000	12 мс	0.01% (Сетевые тайм-ауты)
Go API Gateway + Redis (Проверка Rate Limit)	Атака с ограниченного пула IP	12 000	18 мс	0.00% (Отработка 429 кода)
Python CRM Core (Чтение списка сделок)	Запросы к Rgpool (кэшированные)	3 500	45 мс	0.00%
Python CRM Core (Транзакционные записи)	Смена статусов (FSM валидация)	800	110 мс	0.00%

Тестируемый компонент	Характер нагрузки	Пиковое значение RPS	Среднее время отклика (Latency)	Процент ошибок (Error Rate)
Centrifugo Cluster	Широковещательная рассылка	20 000 conn.	< 50 мс	0.00%

Таблица 3.1 — Результаты нагрузочного тестирования программного комплекса

3.4.3. Оценка отказоустойчивости (Fault Tolerance)

На заключительном этапе тестирования проводилась симуляция аппаратных и программных сбоев (Chaos Engineering) в среде оркестрации Docker Swarm.

1. Отказ узла базы данных: При принудительной остановке мастер-узла PostgreSQL балансировщик Pgpool корректно перевел сессии в режим ожидания до момента автоматического повышения (promotion) реплики до статуса мастера. Потери транзакционных данных не зафиксировано, драйвер asyncpg успешно переподключился к новому пулу.

2. Сбой брокера сообщений: При временной недоступности кластера RabbitMQ, запросы на изменение статуса сделки (выполняемые синхронно) продолжали успешно обрабатываться базой данных. События уведомлений, отправляемые через faststream, буферизировались и были успешно доставлены после восстановления работоспособности RabbitMQ, что подтверждает правильность применения асинхронного, событийно-ориентированного подхода.

3. Падение микросервиса: Имитация аварийного завершения процесса (OOM Kill) одной из реплик микросервиса на базе Python привела к мгновенному перенаправлению трафика шлюзом Go на оставшиеся здоровые реплики. Docker Swarm автоматически

перезапустил упавший контейнер в течение 3 секунд, обеспечив концепцию Zero-Downtime.

Результаты проведенного комплексного тестирования объективно доказывают, что разработанная веб-система полностью соответствует заявленным в первой главе нефункциональным требованиям. Применение микросервисной архитектуры, высокопроизводительного шлюза на Go и строгой алгоритмической фильтрации на Python обеспечило необходимый уровень отказоустойчивости, безопасности и готовности платформы к промышленной эксплуатации в финансовом секторе.

3.4.5. Итоги тестирования

Подводя итоги проведенного цикла испытаний, можно сделать научно и практически обоснованный вывод о том, что спроектированный и реализованный аппаратно-программный комплекс полностью удовлетворяет совокупности функциональных и нефункциональных требований, формализованных на начальных этапах исследования. Комплексная верификация системы подтвердила правильность выбранных архитектурных парадигм и технологического стека.

Во-первых, функциональное тестирование доказало абсолютную математическую и логическую консистентность внедренной модели детерминированного конечного автомата (FSM) в связке с ролевой матрицей доступа (RBAC). Было эмпирически подтверждено, что программная инкапсуляция бизнес-логики на уровне микросервиса (Python/FastAPI) полностью нивелирует риски несанкционированного изменения статусов финансовых заявок. Любые попытки нарушения хронологии транзакций или превышения должностных полномочий алгоритмически блокируются, что обеспечивает требуемый уровень юридической и финансовой прозрачности процессов.

Во-вторых, результаты нагрузочных испытаний объективно верифицировали высокую пропускную способность пограничного периметра сети. Реализация микросервисного шлюза (API Gateway) на компилируемом

языке Go в интеграции с алгоритмом «Скользящего окна» (Sliding Window) на базе кластера Redis продемонстрировала способность системы выдерживать пиковые нагрузки свыше 12 000 запросов в секунду. Жесткое отсечение нелегитимного трафика на уровне шлюза гарантирует защиту транзакционного ядра базы данных (PostgreSQL) от истощения ресурсов (Resource Exhaustion) и предотвращает деградацию производительности (Latency) для авторизованных пользователей.

В-третьих, имитация аппаратных и программных сбоев (методология Chaos Engineering) подтвердила заявленный уровень отказоустойчивости распределенной инфраструктуры. Использование оркестратора Docker Swarm обеспечило автоматическое самовосстановление (Self-healing) упавших контейнеров без прерывания обслуживания (Zero-Downtime). В свою очередь, применение паттерна событийно-ориентированной архитектуры (Event-Driven Architecture) с использованием шины RabbitMQ и фреймворка faststream доказало свою эффективность: временная недоступность отдельных асинхронных узлов не приводит к каскадным сбоям (Cascading Failures) всего программного комплекса.

Таким образом, результаты тестирования позволяют констатировать, что разработанная микросервисная веб-система обладает необходимым запасом прочности, высокой степенью информационной безопасности и масштабируемости. Успешное прохождение всех сценариев верификации свидетельствует о полной готовности программного продукта к интеграции в реальную промышленную среду (Production Environment) для обслуживания трансграничных финансовых операций.

Выводы по третьей главе

В третьей главе выпускной квалификационной работы выполнена программная реализация спроектированной микросервисной архитектуры и проведена комплексная верификация веб-системы. Результаты подтвердили успешное создание отказоустойчивого программного комплекса, готового к эксплуатации в условиях реальных транзакционных нагрузок.

Ядро бизнес-логики реализовано на высокоуровневом языке программирования с применением асинхронного веб-фреймворка, что обеспечило высокую производительность операций ввода-вывода. Ключевым достижением стала строгая алгоритмическая инкапсуляция многомерного конечного автомата (FSM) и матрицы ролевого доступа (RBAC). Использование паттернов внедрения зависимостей, декларативной валидации данных и асинхронных драйверов баз данных гарантировало абсолютную целостность процессов. Доказано, что система программно блокирует любые несанкционированные изменения статусов заявок, исключая влияние человеческого фактора.

Единый микросервисный шлюз (API Gateway) разработан на высокопроизводительном компилируемом языке. Делегирование шлюзу сквозных задач, включая валидацию токенов доступа (JWT) и маршрутизацию трафика, существенно разгрузило транзакционное ядро. Интеграция алгоритма «Скользящее окно» (Sliding Window) на базе распределенного in-memory хранилища сформировала надежный интеллектуальный брандмауэр против сетевых перегрузок, атак прикладного уровня и попыток автоматизированного парсинга.

Разработка клиентской части с применением технологий гибридного рендеринга подтвердила эффективность концепции динамического отображения интерфейса (Role-Based Rendering). Веб-интерфейс превентивно адаптируется под профиль пользователя, алгоритмически скрывая конфиденциальную коммерческую информацию от неавторизованных субъектов, что обеспечивает строгое соблюдение политик информационной безопасности и высокую эргономичность.

Многоуровневое тестирование всесторонне верифицировало заявленные характеристики системы. Нагрузочные испытания подтвердили стабильность шлюза при пиковом трафике без критической деградации времени отклика. Функциональные тесты доказали математическую корректность переходов FSM, а симуляция программно-аппаратных сбоев продемонстрировала способность распределенной инфраструктуры к

автоматическому самовосстановлению без прерывания обслуживания (Zero-Downtime).

Таким образом, реализованный аппаратно-программный комплекс в полной мере удовлетворяет всем выдвинутым функциональным и нефункциональным требованиям. Изоляция бизнес-доменов, внедрение производительного шлюза и многоуровневое разграничение прав позволили создать безопасную, масштабируемую и отказоустойчивую платформу, представляющую собой технически совершенное решение для финансовой отрасли.

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PostgreSQL: Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 06.04.2026).
2. FastAPI [Электронный ресурс] : official documentation. — URL: <https://fastapi.tiangolo.com/> (дата обращения: 06.04.2026).
3. Features [Электронный ресурс] // FastAPI Documentation. — URL: <https://fastapi.tiangolo.com/features/> (дата обращения: 06.04.2026).
4. First Steps [Электронный ресурс] // FastAPI Documentation. — URL: <https://fastapi.tiangolo.com/tutorial/first-steps/> (дата обращения: 06.04.2026).
5. PostgreSQL: The World's Most Advanced Open Source Database [Электронный ресурс]. — URL: <https://www.postgresql.org/> (дата обращения: 06.04.2026).
6. React [Электронный ресурс] : official documentation. — URL: <https://react.dev/> (дата обращения: 06.04.2026).
7. Next.js Docs [Электронный ресурс]. — URL: <https://nextjs.org/docs> (дата обращения: 06.04.2026).
8. Next.js [Электронный ресурс] : The React Framework for the Web. — URL: <https://nextjs.org/> (дата обращения: 06.04.2026).
9. Docker Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/> (дата обращения: 06.04.2026).
10. Get Started with Docker [Электронный ресурс]. — URL: <https://www.docker.com/get-started/> (дата обращения: 06.04.2026).
11. Redis Documentation [Электронный ресурс]. — URL: <https://redis.io/docs/latest/> (дата обращения: 06.04.2026).
12. GitLab CI/CD Documentation [Электронный ресурс] // GitLab. — URL: <https://docs.gitlab.com/ee/ci/> (дата обращения: 06.04.2026).
13. NGINX Documentation [Электронный ресурс]. — URL: <https://nginx.org/en/docs/> (дата обращения: 06.04.2026).
14. Microservices [Электронный ресурс] // Martin Fowler. — URL: <https://martinfowler.com/articles/microservices.html> (дата обращения: 06.04.2026).

1. Role-Based Access Control (RBAC) [Электронный ресурс] // NIST Computer Security Resource Center. — URL: <https://csrc.nist.gov/projects/role-based-access-control> (дата обращения: 06.04.2026).

ПРИЛОЖЕНИЕ А